```python
# Python Program
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

# Take input from user
num = int(input("Enter
    a positive integer: "))
# Check and display result
if is_prime(num):
    print(num, "is a
        a prime number.")
else:
    print(num, "is not
        a prime number.")
```

# STATE BOARD OF TECHNICAL EDUCATION KERALA

## DIRECTORATE OF TECHNICAL EDUCATION

GOVERNMENT OF KERALA

## Diploma in Engineering

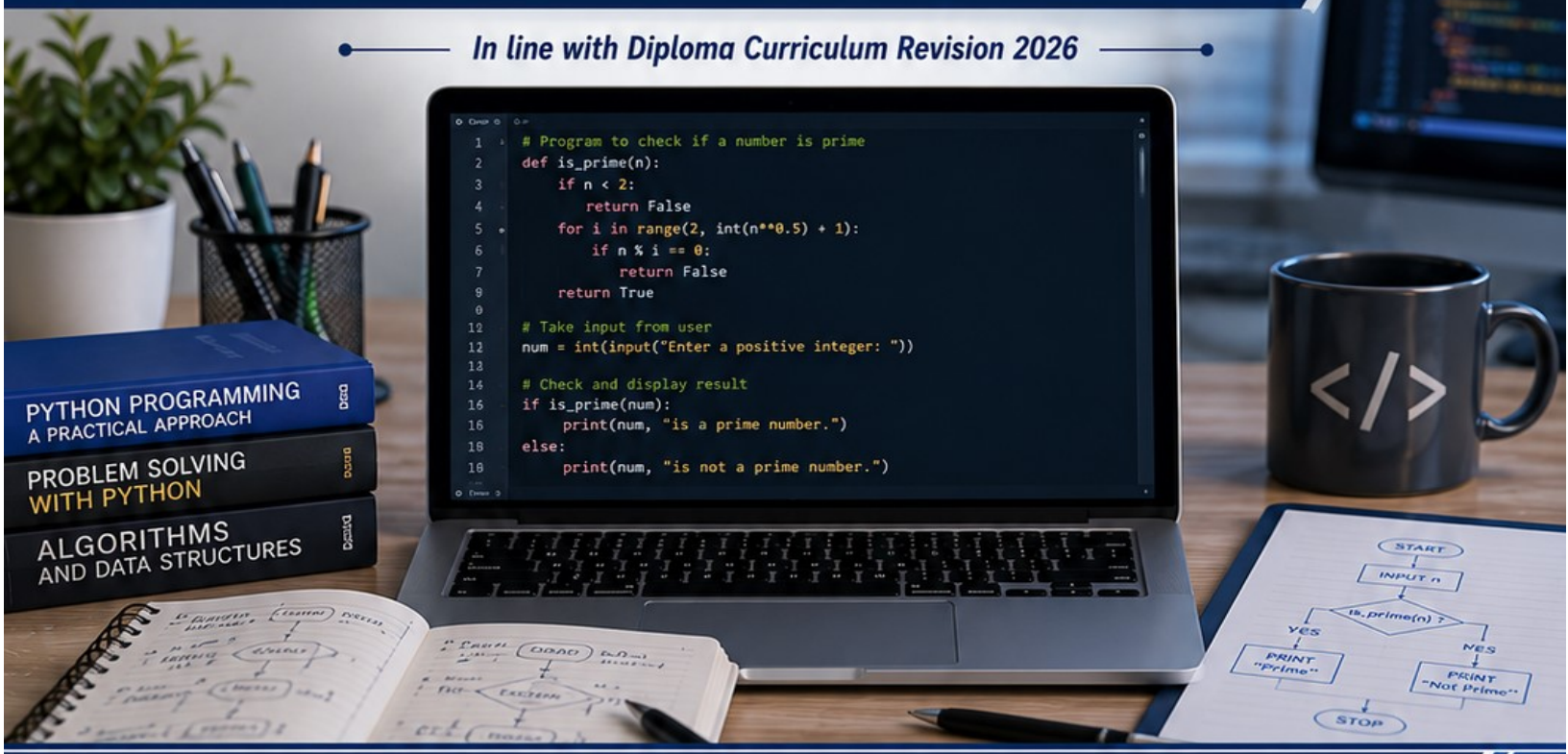
**Semester 1**

# LAB MANUAL

**1131**

# Problem Solving and Python Programming

*In line with Diploma Curriculum Revision 2026*




STATE INSTITUTE OF TECHNICAL TEACHERS TRAINING AND RESEARCH, KALAMASSERY

STATE BOARD OF TECHNICAL EDUCATION KERALA

DIRECTORATE OF TECHNICAL EDUCATION

GOVERNMENT OF KERALA

**Diploma in Computer Engineering**

Semester 1

# LABORATORY MANUAL

## 1131 - Problem Solving and Python Programming

For Diploma Curriculum Revision 2026

**Prepared by:**
**STATE INSTITUTE OF TECHNICAL TEACHERS TRAINING AND RESEARCH, KALAMASSERY**

# PREFACE

This laboratory manual is prepared for first-semester diploma students of all computer branches as part of the Problem Solving and Python Programming course under the Diploma Curriculum Revision 2026. The Problem Solving and Python Programming Laboratory Manual has been prepared to provide students with a systematic and practical approach to learning the fundamentals of problem solving and Python programming. Aims to bridge the gap between theoretical concepts and their practical implementation. Through a series of carefully structured experiments, students are introduced to algorithm design, flowchart development, Python programming constructs, data handling techniques, and modular programming concepts.

Each experiment is aligned with the Course Outcomes (COs) and Program Outcomes (POs) of the curriculum, following a competency-based, student-centered approach. Students are expected to observe carefully, analyse results, and document their work in a structured manner.

# PROGRAM OUTCOMES

| POs | Statements |
|---|---|
| PO1 | Apply knowledge of mathematics, natural sciences, engineering fundamentals, and an engineering specialization (as specified in DK1 to DK4) to a wide range of practical procedures and practices |
| PO2 | Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis specific to their field of activity (DK1 to DK4). |
| PO3 | Design solutions for well-defined technical problems and assist in the design of systems, components, or processes to meet specified needs, with appropriate consideration for public health and safety, as well as cultural, societal, and environmental factors (DK5). |
| PO4 | Conduct investigations of well-defined problems by locating and searching relevant codes and catalogues, and by conducting standard tests and measurements (DK8). |
| PO5 | Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems, while being aware of their limitations (DK2 and DK6). |
| PO6 | When solving well-defined engineering problems, evaluate sustainable development impacts to: society, the economy, sustainability, health and safety, legal frameworks, and the environment (DK1, DK5, and DK7). |
| PO7 | Understand and commit to professional ethics and the norms of technician practice, including compliance with relevant laws. Demonstrate an understanding of the need for diversity and inclusion (DK9). |
| PO8 | Function effectively as an individual, and as a member or leader in diverse and inclusive teams, in multidisciplinary face-to-face/remote/distributed settings (DK9). |
| PO9 | Communicate effectively and inclusively on well-defined engineering activities with the engineering community and society at large by comprehending the work of others, documenting one's own work, and giving and receiving clear instructions. |
| PO10 | Demonstrate awareness of engineering management principles as a member or leader of a technical team and apply them to manage projects in multidisciplinary environments. |
| PO11 | Recognize the need for, and have the ability for independent updating in the face of specialized technical knowledge (DK8). |

# KNOWLEDGE AND ATTITUDE PROFILE

| DKs | Statements |
|---|---|
| DK1 | A descriptive, formula-based understanding of the natural sciences applicable in a sub-discipline and awareness of directly relevant social sciences. |
| DK2 | Procedural mathematics, numerical analysis, statistics applicable in a sub discipline. |
| DK3 | A coherent procedural formulation of engineering fundamentals required in an accepted sub-discipline. |
| DK4 | Engineering specialist knowledge that provides the body of knowledge for an accepted sub-discipline. |
| DK5 | Knowledge that supports engineering design and operations based on the techniques and procedures of a practice area. |
| DK6 | Codified practical engineering knowledge in recognized practice area. |
| DK7 | Knowledge of issues and approaches in engineering technician practice, such as public safety and sustainable development. |
| DK8 | Engagement with the current technological literature of the practice area. |
| DK9 | Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes. |

# COURSE DETAILS

| ITEM | DETAILS | |
|---|---|---|
| Programme | Diploma in Computer Engineering | |
| Semester | Semester 1 | |
| Course Name | Problem Solving and Python Programming | |
| Course Code | 1131 | |
| Course Type | Practicum | |
| Credits | 4.5 | |
| Hours/Week | 6 | |
| Total Hours | 90 | |
| Curriculum | 2026 | |
| Prepared By | Syam Kumar K S | Sanjay Krishnadas |
| Institution | Govt Polytechnic College Vandiperiyar | Govt Polytechnic College Perumbavoor |

# COURSE OUTCOME MATRIX

| CO | Course Outcome | Blooms Taxonomy Level |
|---|---|---|
| **CO1** | Acquire foundational logical reasoning skills by designing flowcharts and algorithms to develop Python programs that effectively incorporate variables, fundamental data types, and standard input/output operations. | A |
| **CO2** | Optimize program execution through the strategic use of control structures and loop control statements. | A |
| **CO3** | Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations, built-in methods to solve real-world programming problems | A |
| **CO4** | Design and develop modular programs using functions, modules, and packages with proper scope management and argument passing techniques | A |

| COURSE ARTICULATION MATRIX | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Course Outcomes** | PROGRAM OUTCOMES | | | | | | | | | | |
| | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 |
| **CO1** | 3 | 3 | 2 | 2 | 2 | - | - | - | - | - | 3 |
| **CO2** | 3 | 3 | 3 | 2 | 2 | - | - | - | - | - | 3 |
| **CO3** | 3 | 3 | 3 | 3 | 3 | - | - | - | - | - | 3 |
| **CO4** | 3 | 2 | 3 | 3 | 3 | - | - | - | - | - | 3 |
| **Total** | 12 | 11 | 11 | 10 | 10 | 0 | 0 | 0 | 0 | 0 | 12 |
| **Correlation level** | 3 | 2.75 | 2.75 | 2.5 | 2.5 | 0 | 0 | 0 | 0 | 0 | 3 |
| *Legends: 1 - Low; 2 - Medium; 3 - High; No Correlation - "-"* | | | | | | | | | | | |

| Teaching Scheme/ Week | | | | Self-learning (SS)/ Week | Total Hours/ Semester | Credits C | Examination Scheme | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| L | T | P | R | | | | Theory | | | Practical | | | Total |
| 3 | 0 | 3 | 0 | 6 | 90 | 4.5 | CIE | ESE | Total | CIE | ESE | Total | |
| | | | | | | | 20 | 60 | 80 | 20 | 40 | 60 | 140 |
| *L: Lecture (One unit is of one-hour duration with one credit), T: Tutorial (One unit is of one-hour duration with one credit), P: Practical (One unit is of one-hour duration with 0.5 credit), S: Project (One unit is of one-hour duration with 0.5 credit)*<br>*SS: Self-Learning, CIE: Continuous Internal Evaluation, ESE: End Semester Examination* | | | | | | | | | | | | | |

## GENERAL INSTRUCTIONS

1. Rough record and Fair record are needed to record the experiments conducted in the laboratory.
2. Rough records are needed to be certified immediately on completion of the experiment.
3. Fair records are due at the beginning of the next lab period.
4. Fair records must be submitted as neat, legible, and complete.

## INSTRUCTIONS FOR WRITING THE FAIR RECORD

In the fair record, the index page should be filled properly by writing the corresponding experiment number, experiment name, date on which it was done and the page number.

On the **right-side** page of the record, the following has to be written:

1. **Title**: The title of the experiment should be written on the page in capital letters.
2. In the left top margin, **experiment number** and **date** should be written.
3. **Practical significance/Aim**: The purpose of the experiment should be written clearly.
4. **Relevant course outcome**: The outcome to be attained at the end of the experiment has to be mentioned clearly.
5. **Theory**: Simple working of the circuit/experimental set up/algorithm should be written.
6. **Resources required**: A list of the Apparatus/Tools/ Equipment/Components used for doing the experiment should be entered.
7. **Precautions to be followed**: The detailed description of the precautions to be followed while performing the experiments should be clearly mentioned.

8. **Procedure**: Steps for doing the experiment and recording the readings should be briefly described.
9. **Results**: The results of the experiment must be summarized in writing and should be fulfilling the aim.
10. **Interpretation of the result**: The interpretations obtained after performing the experiments and viewing/calculating the observations should be listed.
11. **Conclusion**: The statement on the completion of the experiment should be mentioned.

On the **left-side** page of the record following has to be recorded:

1. **Algorithm**: Neatly write the algorithm.
2. **Flowchart**: Neatly draw the flowchart
3. **Observations**:
   a. Data should be clearly recorded using Tabular Columns.
   b. The unit of the observed data should be clearly mentioned.
   c. Relevant calculations should be shown. If repetitive calculations are needed, only show a sample calculation and summarize the others in a table.
4. **Graphs**: Graphs can be used to present data in a form that shows the results obtained, as one or more of the parameters are varied. A graph has the advantage of presenting large amounts of data in a concise visual form. The graph should be in a square format.

# GENERAL GUIDELINES TO TEACHERS

The experiments are developed such that the major practical learning outcomes are achieved. Each experiment is designed to achieve the course outcomes mentioned in the curriculum.

1. Teachers should provide the guideline with demonstration of experiment to the students with all features.
2. The teacher shall explain prior concepts to the students before the starting of each practical session.
3. Involve students in the performance of each experiment.
4. Teachers are expected to share the skills and competencies to be developed in the students.
5. Teachers should ensure that the respective skills and competencies are developed in the students after the completion of the practical exercise.
6. Teachers should give opportunities to students for hands-on experience after the demonstration.
7. Teachers may provide additional knowledge and skills to the students even though not covered in the manual but are expected from the students by the industry.
8. Teachers may suggest additional related literature of technical papers/reference books.
9. Finally give practical assignments and assess the performance of the students based on tasks assigned to check whether it is as per the instructions.
10. The teacher is expected to refer to the complete curriculum document and follow guidelines for implementation.
11. For difficult practical, if required, the teacher could provide demonstration of the practical emphasizing of skills which the student should achieve.
12. Assess the skill achievement of the students and COs after each unit.
13. The assessment should be performed on a regular basis.

# GENERAL GUIDELINES TO STUDENTS

1. Listen carefully to all the information regarding curriculum, its course outcomes and major learning outcomes, equipment and instruments in the laboratory and the method of assessment.
2. Read the writeup of each experiment to be performed, a day advance.
3. Be **punctual** for your laboratory sessions.
4. It is mandatory to come to the lab in a formal dress and wear your ID cards.
5. Mobile phones should be switched off in the lab. Keep bags in the bag rack.
6. Sign in the log register as soon as you enter the lab and strictly observe your lab timings.
7. Strictly follow the written and verbal instructions given by the teacher / Lab Instructor. If you do not understand the instructions, the handouts and the procedures, ask the instructor or teacher.
8. **Never work alone!** You should be accompanied by your laboratory partner and / or the instructors / teaching assistants all the time.
9. Organize the work in a group and record all the observations.
10. Understand the practical implications of the experiments.
11. Students should develop maintenance and troubleshooting skills.
12. Keep the labs clean at all times; no food and drinks allowed inside the lab.
13. Intentional misconduct will lead to expulsion from the lab.
14. After completion of the experiment, return the bread board, trainer kits, wires, CRO probes and other components to lab staff. Do not take any item from the lab without permission.
15. Observation books and lab records should be carried to each lab. Readings of current lab experiments are to be entered in the observation book and previous lab experiments should be written in the lab record book. Both the books should be corrected by the faculty in each lab.

# SAFETY INSTRUCTIONS

1. Do not wear loose-fitting clothing or jewelry in the lab. Rings and necklaces are excellent conductors of electricity.
2. Sandals or open-toed shoes are NOT allowed.
3. Do not handle any equipment without reading the safety instructions. Read the handout and procedures in the lab manual before starting the experiments.
4. Setup the circuit and perform a careful circuit check before applying power. Do not make circuit changes or perform any wiring when power is on.
5. Avoid contact with energized electrical circuits.
6. Do not insert connectors forcefully into the sockets.
7. **NEVER** try to experiment with the power from the wall plug.
8. Immediately report dangerous or exceptional conditions to the Lab instructor / teacher: Equipment that is not working as expected, wires or connectors are broken, the equipment that smells or "smokes". If you are not sure what the problem is or what's going on, switch off the Emergency shutdown.
9. Never use damaged instruments, wires or connectors. Hand over these parts to the Lab instructor/Teacher.
10. Be sure of the location of fire extinguishers and first aid kits in the laboratory.
11. Handling of Semiconductor Components: Sensitive electronic circuits and electronic components have to be handled with great care. The inappropriate handling of electronic components can damage or destroy the devices. The devices can be destroyed by driving to high currents through the device, by overheating the device, by mixing up the polarity, or by electrostatic discharge (ESD). Therefore, always handle the electronic devices as indicated by the handout, the specifications in the data sheet or other documentation.

# ASSESSMENT METHODOLOGY

| Mode | Attendance | CA1 | CA2 | CA3 | CA4 | CA5 | CA6 | ESE | |
|---|---|---|---|---|---|---|---|---|---|
| | | Self learning activities from each module | Conti. evaluation | Practical test (first half of experiments) | Practical test (second half of experiments) | Series exam (2 modules) | Series exam (2 modules) | Written exam (theory) | Practical |
| Duration | | | | 3 hours | 3 hours | 2 hours | 2 hours | 2.5 hours | 3 hours |
| Exam mark | | 10 | 50 | 40 | 40 | 50 | 50 | 60 | 40 |
| Converted to | | | 10 | 10 | 10 | 10 | 10 | | |
| Marks | 5 | 5 | 10 | 10 | | 10 | | 60 | 40 |
| Total | 40 | | | | | | | 100 | |
| Tentative schedule | End of semester | By 14th week | 14th week | 7th week | 14th week | 8th week | 15th week | End of semester | End of semester |

**Scheme of Evaluation - Continuous Internal Evaluation**

| Sl. No. | Criteria | Description | Marks |
|---|---|---|---|
| 1 | Preparation & Punctuality | Attendance, timely submission of record, pre-lab preparation, understanding of experiment objectives and theory. | 10 |
| 2 | Experimental Setup & Procedure | Ability to identify components/equipment, correct circuit connections/setup, systematic and safe execution of the procedure. | 10 |
| 3 | Observation & Data Recording | Accuracy and neatness of readings, correct use of units, tabulation, and systematic recording of results. | 5 |
| 4 | Analysis & Result Interpretation | Proper calculations, graphical representation, result verification, discussion, and logical conclusion. | 10 |
| 5 | Viva Voce / Concept Understanding | Clarity of theoretical concepts, explanation of procedure, ability to answer questions related to the experiment. | 10 |
| 6 | Workmanship, Discipline & Teamwork | Laboratory discipline, care of instruments, teamwork, attitude, and safety practices. | 5 |
| TOTAL | | | 50 |

**Scheme of Evaluation - Practical Test & End Semester Evaluation**

| Category | Things to evaluate | Marks |
|---|---|---|
| Write-up / Procedure (before experiment) | Aim, circuit diagram/flowchart, stepwise procedure. | 10 |
| Experiment Setup & Execution | Correct circuit connections / coding / setup. Handling of instruments/software. Accuracy of execution. | 10 |
| Observation & Result / Output | Proper tabulation of readings. Calculations/graphs/program output. Correctness of result / expected outcome. | 8 |
| Viva Voce | Conceptual understanding of experiment. Related theoretical knowledge. | 8 |
| Record | Completion & neatness of practical record. Proper certification by faculty. | 4 |
| TOTAL | | 40 |

**Scheme of Evaluation - Open-ended experiment**

| Category | Things to evaluate | Marks |
|---|---|---|
| Originality and Relevance of the Idea | Assesses how creative, innovative, and relevant the chosen experiment is to the course objectives. | 10 |
| Clarity of Objectives and Experimental Plan | Evaluates how clearly the student defines the aim, scope, and systematic plan of the experiment. | 10 |
| Correctness of Execution and Data Recording | Checks the accuracy, safety, and neatness in performing the experiment and recording observations. | 10 |
| Analysis, Interpretation, and Presentation of Results | Measures the student's ability to analyze data, interpret outcomes, and present results logically. | 10 |
| Teamwork and Innovation Demonstrated | Judges the level of collaboration, initiative, and creativity shown during the experiment. | 10 |
| TOTAL | | 50 |

# CONTENT PAGE


| Sl. No. | Title of the practical | Mapped CO | Page No. | Executed date | Remarks |
|---|---|---|---|---|---|
| 1 | Algorithm and Flowchart | CO1 | | | |
| 2 | Installation of Python and an IDE | CO1 | | | |
| 3 | Introduction to Python Programming | CO1 | | | |
| 4 | Decision Control Structures | CO2 | | | |
| 5 | Selection-based control structure | CO2 | | | |
| 6 | Loop Control Structure | CO2 | | | |
| 7 | String Manipulations | CO3 | | | |
| 8 | List Operations | CO3 | | | |
| 9 | Dictionary Operations | CO3 | | | |
| 10 | Set Operations | CO3 | | | |
| 11 | Built-in Functions | CO4 | | | |
| 12 | User-Defined Functions | CO4 | | | |

| Experiment no. : 1 | Date: |
|---|---|
| **Algorithm and flowchart for Simple Problems** | |

## PROBLEM STATEMENTS

1. A bank wants to develop a system to calculate the simple interest for customers based on the deposited amount, rate of interest, and time period. Design an algorithm and flowchart to calculate and display the simple interest.
2. A school wants to create a program to identify whether a student's roll number is odd or even for classroom arrangement purposes. Design an algorithm and flowchart to check whether a given number is odd or even.
3. A teacher wants to display natural numbers from 1 up to a specified limit N for teaching counting concepts in class. Design an algorithm and flowchart to print natural numbers up to N.

## PRACTICAL SIGNIFICANCE

*This experiment helps students develop logical thinking and problem-solving skills by designing algorithms and flowcharts for simple real-life problems. It provides a basic understanding of program planning, sequence of operations, and decision-making techniques used in computer programming. The experiment also improves students' ability to analyze problems systematically before implementing them in a programming language*

## RELEVANT PROGRAM OUTCOMES(POs)

PO2 – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis specific to their field of activity.

PO3 – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes to meet specified needs.

PO5 – Apply appropriate techniques, resources, and modern computing, engineering, and IT tools to well-defined engineering problems, while being aware of their limitations.

PO9 – Communicate effectively and inclusively on well-defined engineering activities by comprehending the work of others, documenting one's own work, and giving and receiving clear instructions.

## RELEVANT COURSE OUTCOMES(COs)

CO1: Acquire foundational logical reasoning skills by designing flowcharts and algorithms and develop Python programs that effectively incorporate variables, fundamental data types, and standard input/output operations.

## THEORY

### Introduction to Algorithm

An **algorithm** is a step-by-step procedure used to solve a problem or perform a task. It is written in a simple and logical sequence so that a computer or a person can understand and execute it easily.

### General Algorithm Steps

1. **Input** — Define what data or values the algorithm receives
2. **Initialization** — Set up variables, counters, or data structures needed
3. **Condition Check** — Evaluate if a condition is true or false (decision point)
4. **Process / Operation** — Perform the core logic (calculations, comparisons, transformations)
5. **Iteration** — Repeat a block of steps using a loop (if needed)
6. **Update** — Modify variables or data based on the operation result
7. **Exit Condition** — Determine when to stop the loop or process
8. **Output** — Return or display the final result

### Characteristics of an Algorithm

- Simple and easy to understand
- Step-by-step representation
- Finite number of steps
- Produces correct output
- Language independent

**Introduction to Flowchart**

A flowchart is a graphical representation of an algorithm using symbols and arrows. It visually shows the sequence of operations in a program.

**Common Flowchart Symbols**

| Symbol | Name | Function |
|---|---|---|
|  | Start/end | An oval represents a start or end point |
|  | Arrows | A line is a connector that shows relationships between the representative shapes |
|  | Input/Output | A parallelogram represents input or output |
|  | Process | A rectangle represents a process |
|  | Decision | A diamond indicates a decision |

**ALGORITHM/FLOWCHART**

**Problem 1: Simple Interest Calculation**

Simple Interest is the interest calculated only on the principal amount for a fixed rate and time period.

**Formula** $SI = \frac{P \times R \times T}{100}$

Where:

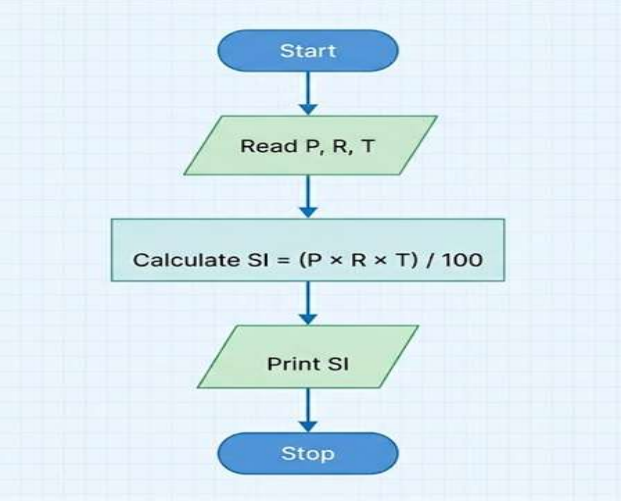
- **P** = Principal Amount
- **R** = Rate of Interest
- **T** = Time
- **SI** = Simple Interest

The algorithm takes the principal amount, rate, and time as input and calculates the simple interest using the formula.

**Algorithm**

1. Start
2. Read P, R, T
3. Calculate SI = (P × R × T) / 100
4. Print SI
5. Stop

**Flowchart**




**Problem 2: Odd or Even Number Check**

A number is:

- **Even** if it is divisible by 2
- **Odd** if it is not divisible by 2

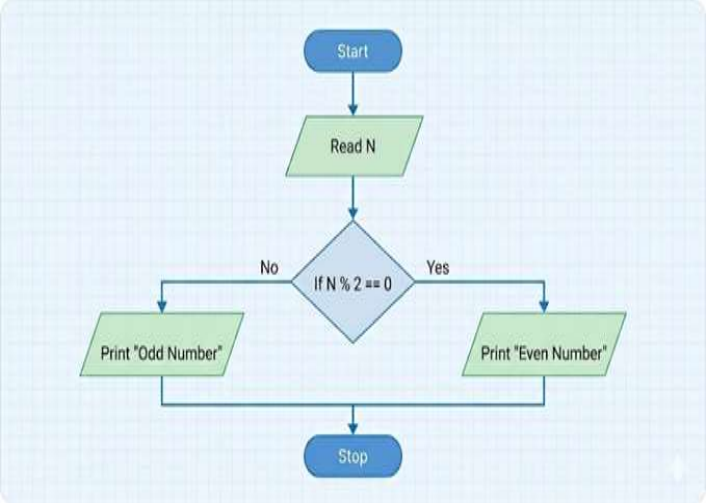
The modulus operator (%) gives the remainder after division.

If: $N mod 2 = 0$

then the number is even; otherwise it is odd.

**Algorithm**

1. Start
2. Read N
3. If N % 2 == 0
4. Print “Even Number”
5. Else
6. Print “Odd Number”
7. Stop




**Flowchart**

**Problem 3: Print Natural Numbers up to N**

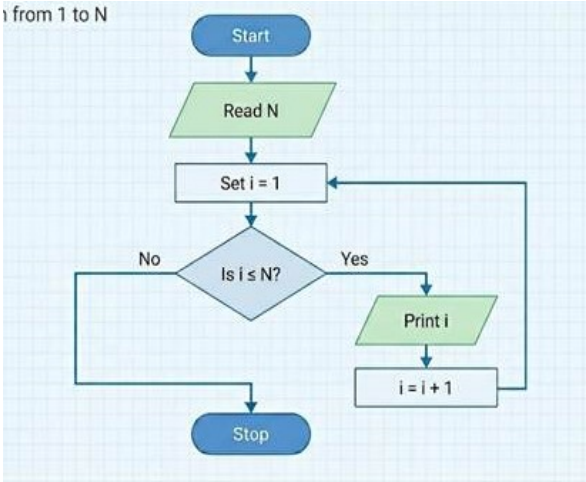
Natural numbers are positive integers starting from 1. This program prints numbers from 1 to N using a loop.A loop repeatedly executes a set of statements until a condition becomes false.

**Algorithm**

1. Start
2. Read N

3. Set i = 1
4. Repeat steps 5 and 6 while i ≤ N
5. Print i
6. i= i +1
7. Stop

**Flowchart**




## RESULTS

The algorithm and flowchart for all problems were designed and executed successfully

## PRACTICAL RELATED QUESTIONS

1. ***What is an algorithm?***

   *An algorithm is a step-by-step procedure used to solve a problem.*

2. ***What is a flowchart?***

   *A flowchart is a graphical representation of an algorithm using symbols.*

3. ***Which symbol is used for decision making in a flowchart?***

   *Diamond symbol.*

4. ***Why are algorithms important?***

   *Algorithms help in understanding program logic clearly before coding.*

5. ***What is the first step in developing a program?***

*Understanding the problem and designing the algorithm.*

## SELF- LEARNING COMPONENTS

- Draw different flowchart symbols and their use.
- Students design simple algorithms and flowcharts for daily life problems such as calculating total marks / finding the largest of two numbers/Simple formula solving

## ADDITIONAL PROBLEM STATEMENTS

1. A company wants to calculate the monthly salary of its employees based on the number of working days and daily wage. Design an algorithm and flowchart to calculate and display the employee's salary.
2. A water supply department wants to determine whether a water tank is full, partially filled, or empty based on current water level readings. Design an algorithm and flowchart to display the tank status.
3. A data-entry system needs to determine the number of digits in an identification number. Design an algorithm and flowchart to count and display the number of digits.

## REFERENCES


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Prelude to Programming : Concepts & Design | Venit, S & Drake E | AddisonWesley (Pearson) |
| | Web references | | |
| 2 | https://www.geeksforgeeks.org/dsa/difference-between-algorithm-and-flowchart/ | | |

| Experiment no. : 2 | Date: |
|---|---|
| **Installation of Python and IDEs** | |

## PRACTICAL SIGNIFICANCE

*This experiment helps students understand the process of installing Python and configuring different IDEs required for program development. It provides practical knowledge of setting up a programming environment for writing, executing, and debugging Python programs efficiently.*

## RELEVANT PROGRAM OUTCOMES(POs)

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems, while being aware of their limitations.

**PO11** – Recognize the need for, and have the ability for independent updating in the face of specialized technical knowledge.

**PO9** – Communicate effectively and inclusively on well-defined engineering activities by comprehending and documenting work.

## RELEVANT COURSE OUTCOMES(COs)

CO1 : Acquire foundational logical reasoning skills by designing flowcharts and algorithms and develop Python programs that effectively incorporate variables, fundamental data types, and standard input/output operations

## THEORY

### Introduction to Python

Python is a high-level, interpreted, and user-friendly programming language widely used for software development, web applications, data analysis, artificial intelligence, and automation. It has a simple syntax that makes programming easier for beginners and professionals. Python programs are executed line by line using the Python interpreter.

**Features of Python**

- Simple and easy to learn
- Interpreted language
- Platform independent
- Supports object-oriented programming
- Large standard library
- Open-source and free to use

**Introduction to IDE**

An Integrated Development Environment (IDE) is software that provides tools for writing, editing, running, and debugging programs in a single interface. IDEs make programming easier by offering features like syntax highlighting, auto-completion, error detection, and debugging support.

**Common Python IDEs**

- **IDLE –** Default Python IDE included with Python installation
- **PyCharm –** Professional IDE with advanced features
- **Visual Studio Code (VS Code) –** Lightweight editor with Python extensions
- **Jupyter Notebook –** Interactive environment mainly used for data science and learning

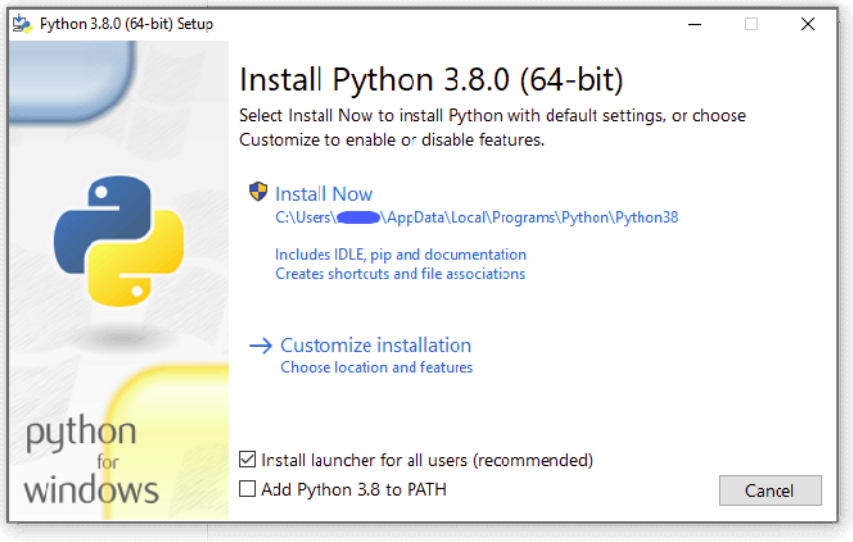
**PROCEDURE**

**Procedure for Installation of Python in Windows**

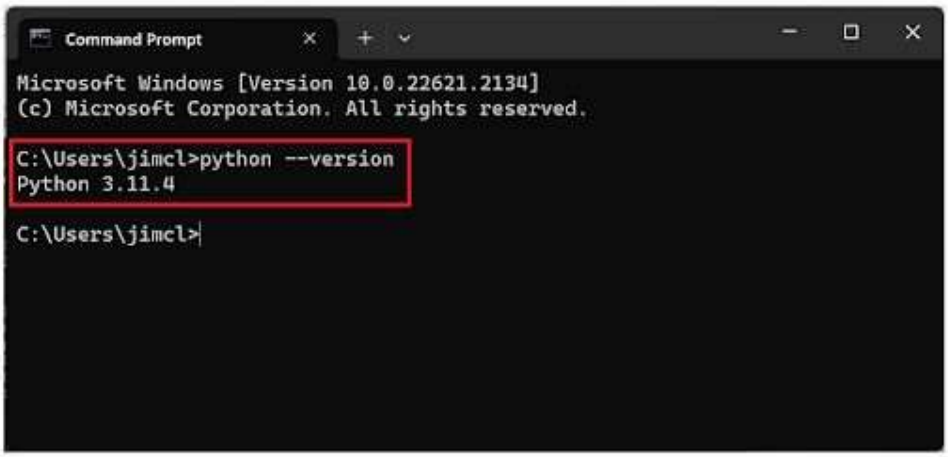
1. Open a web browser and visit the official Python website:
   https://www.python.org/downloads/
2. Download the latest version of Python for Windows.




3. Open the downloaded installer file.
4. Select the option **"Add Python to PATH"**.




5. Click **Install Now** to begin installation.
6. Wait for the installation process to complete.
7. Click **Close** after successful installation.
8. Open **Command Prompt** and type: python --version
   to verify the installation.

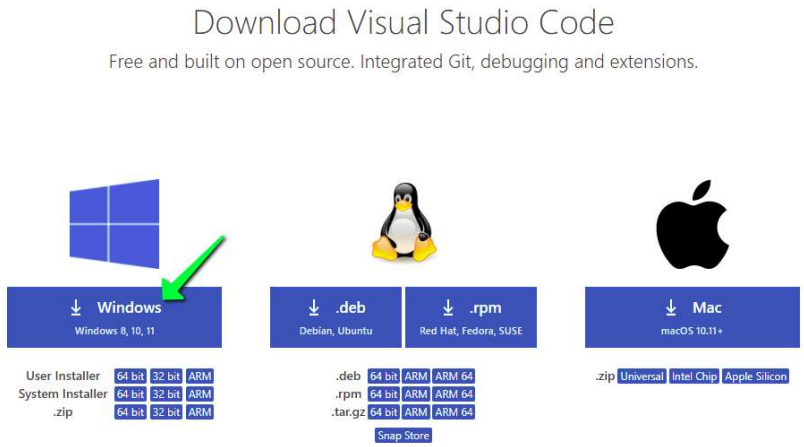



**Procedure for Installation of Python in Ubuntu**

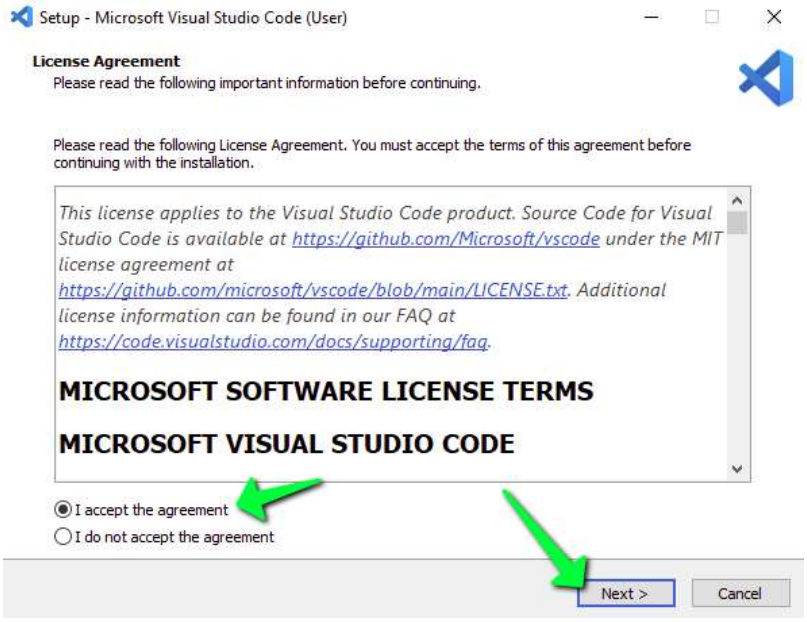
1. Open the **Terminal** in Ubuntu.
2. Update the package list using the command: sudo apt update
3. Install Python using the command: sudo apt install python3
4. Press **Y** and press **Enter** to continue installation.
5. After installation, verify Python using: python3 --version

**Procedure for Installation of Python IDE (VS Code) in Windows**

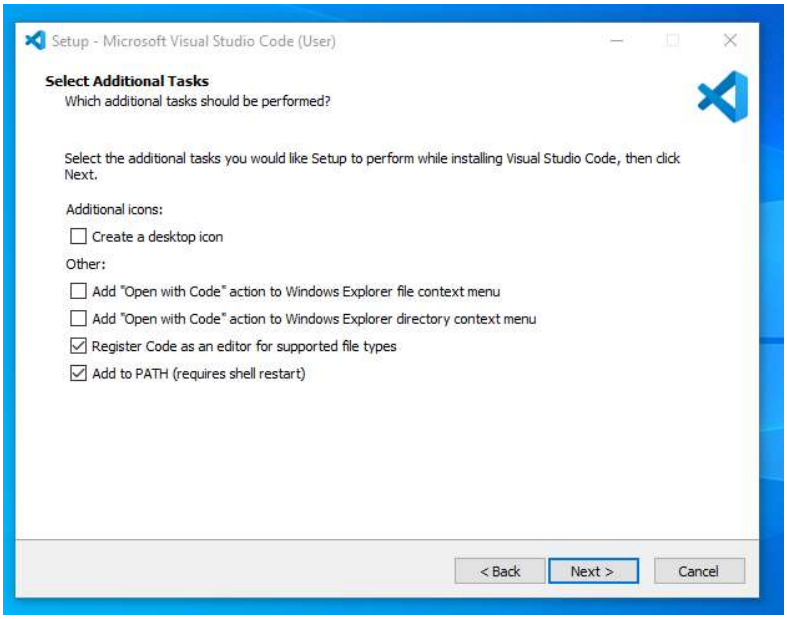
1. Open a web browser and visit: https://code.visualstudio.com/
2. Download the VS Code installer for Windows.




3. Run the installer and click **Next**.
4. Accept the license agreement and continue installation.

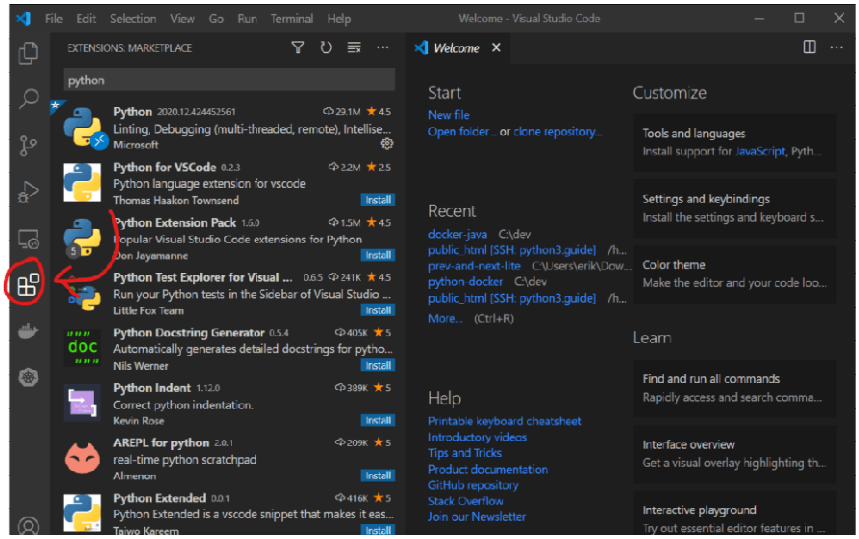



5. Select required options such as:
   - Add to PATH
   - Create Desktop Icon




6. Click **Install** and wait for completion.
7. Open VS Code after installation.
8. Install the Python Extension:
   - Open VS Code
   - Click **Extensions**

- Search for **Python**
- Install the extension developed by Microsoft




9. Verify the setup by creating and running a simple Python program.

**Procedure for Installation of Python IDE (VS Code) in Ubuntu**

1. Open Terminal.
2. Install VS Code using: sudo snap install code --classic
3. Wait for installation to complete.
4. Open VS Code from the Applications menu or by typing: code
5. Open the **Extensions** section.
6. Search for a Python extension and install it.
7. Create a new Python file and run a sample program to verify the IDE setup.

## RESULTS

Python and the Python IDE were successfully installed and configured in both Windows and Ubuntu operating systems.

## PRACTICAL RELATED QUESTIONS

1. ***What is an IDE?***

   *An IDE (Integrated Development Environment) is software used to write, edit, run, and debug programs.*

2. ***What is the purpose of "Add Python to PATH" during installation?***

   *It allows Python commands to be executed from the command prompt or terminal.*

3. ***Name any two Python IDEs.***

   *IDLE and Visual Studio Code (VS Code).*

4. ***Which command is used to check the Python version in Windows?***

   *python --version*

5. ***Which command is used to install Python in Ubuntu?***

   *sudo apt install python3*

## SELF- LEARNING COMPONENTS

- List five features of Python 2.7 and Python 3
- Compare Python IDLE, PyCharm, and VS Code features

## REFERENCES


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Introduction to Computing and Problem Solving using Python | E. Balagurusamy | McGraw Hill Education Pvt Ltd |
| | Web references | | |
| 2 | https://docs.python.org/3/using/windows.html | | |
| 3 | https://www.youtube.com/watch?v=7AEbaAxC7bA | | |

| Experiment no. : 3 | Date: |
|---|---|
| **Introduction to Python Programming** | |

## PROBLEM STATEMENTS

1. A weather monitoring system records temperature in degrees Celsius, but international tourists require the temperature in Fahrenheit[ F=(C×59)+32]. Write a Python program to convert temperature from Celsius to Fahrenheit and display the result.
2. A company wants to calculate the monthly salary of an employee based on the number of working days and daily wage. Write a Python program to calculate and display the salary.
3. A school wants to calculate the percentage obtained by a student in five subjects. Write a Python program to calculate and display the percentage.

## PRACTICAL SIGNIFICANCE

*These programs demonstrate the application of Python in solving real-world problems involving mathematical calculations and data processing. They help students understand the use of variables, input/output operations, arithmetic expressions, and problem-solving techniques in programming.*

## RELEVANT PROGRAM OUTCOMES (POs)

**PO1** – Apply knowledge of mathematics, natural sciences, engineering fundamentals, and an engineering specialization to practical procedures and practices.

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

## RELEVANT COURSE OUTCOMES(COs)

CO1: Acquire foundational logical reasoning skills by designing flowcharts and algorithms and develop Python programs that effectively incorporate variables, fundamental data types, and standard input/output operations

## THEORY

Python is a high-level, interpreted, and general-purpose programming language developed by Guido van Rossum and released in 1991. Python is an interpreted language, programs are executed line by line, making debugging easier.Python provides several built-in data types, including integers (int) for whole numbers, floating-point numbers (float) for decimal values, strings (str) for text, booleans (bool) for logical values (True or False).

Operators in Python are special symbols used to perform operations on variables and values. Arithmetic operators such as +, -, *, /, %, //, and ** are used for mathematical calculations. Comparison operators like ==, !=, >, <, >=, and <= are used to compare values and return Boolean results. Logical operators such as and, or, and not combine or modify logical conditions. Python also provides assignment operators, membership operators, identity operators, and bitwise operators to perform various programming tasks efficiently. Operators play a crucial role in decision-making, calculations, and data manipulation within programs.

Input and output statements enable communication between the user and the program. The input() function is used to accept data from the user during program execution. Since the input received is treated as a string by default, functions such as int() and float() are used to convert the input into numeric data types when required. The print() function is used to display messages, results, and other information on the screen. Python supports different output formatting techniques, including string concatenation, the format() method, and formatted string literals (f-strings), which make program outputs more readable and user-friendly.

## ALGORITHM/FLOWCHART

**Problem 1:**

1. Start
2. Input temperature in Celsius (C)

3. Calculate Fahrenheit using the formula:F=(C×95)+32
4. Display the temperature in Fahrenheit (F)
5. Stop

**Problem 2:**

1. Start
2. Input number of working days
3. Input daily wage
4. Calculate salary :Salary=Working Days×Daily
5. Display salary
6. Stop

**Problem 3:**

1. Start
2. Input marks of five subjects M1,M2,M3,M4,M5
3. Calculate total marks:Total Marks =M1+M2+M3+M4+M5
4. Calculate percentage Percentage=(Total Marks/ 500)×100
5. Display percentage
6. Stop

**PROGRAM CODE**

**Problem 1: Celsius to Fahrenheit Conversion**

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9/5) + 32
print("Temperature in Fahrenheit =", fahrenheit)
```

**Problem 2: Employee Salary Calculation**

```
days = int(input("Enter number of working days: "))
daily_wage = float(input("Enter daily wage: "))
salary = days * daily_wage
print("Monthly Salary =", salary)
```

**Problem 3: Student Percentage Calculation**

```
m1 = float(input("Enter marks of Subject 1: "))
m2 = float(input("Enter marks of Subject 2: "))
m3 = float(input("Enter marks of Subject 3: "))
m4 = float(input("Enter marks of Subject 4: "))
m5 = float(input("Enter marks of Subject 5: "))
total = m1 + m2 + m3 + m4 + m5
percentage = (total / 500) * 100
print("Total Marks =", total)
print("Percentage =", percentage, "%")
```

**PROCEDURE**

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.
   ```
   python <filename>.py
   ```
6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

**RESULTS**

The Python programs were executed successfully, and the required outputs were obtained accurately for the given inputs.

**CONCLUSION**

Python is a powerful, easy-to-learn programming language widely used in various fields of computing. Understanding data types, operators, and input/output statements forms the foundation of Python programming and helps in developing efficient programs for solving real-world problems.

## PRACTICAL RELATED QUESTIONS

1. ***What is the extension of a Python file?***

   *.py*

2. ***What is the purpose of the input() function?***

   *It is used to accept input from the user.*

3. ***What is the purpose of the print() function?***

   *It is used to display output on the screen.*

4. ***What is the difference between int() and float()?***

   *int() converts data into integers, while float() converts data into decimal numbers.*

5. ***What is the default data type returned by input()?***

   *String (str).*

## SELF- LEARNING COMPONENTS

1. Write the variable declaration rules in Python with a suitable example
2. List the different operators in Python and find out their precedence
3. Write Python programs in online Python Compilers (onlineGDB/ Programiz/ Nextleap etc)
   - ○ Write simple programs using variables and operators
   - ○ Document common errors and their solutions.

## ADDITIONAL PROBLEM STATEMENTS

1. A supermarket wants to generate the total bill amount for a customer based on the quantity and price of purchased items. Write a Python program to calculate and display the total bill.

2. A landscape designer wants to calculate the area and circumference of a circular garden using its radius. Write a Python program to calculate and display the results
3. A foreign exchange center wants to convert Indian Rupees into US Dollars using the current exchange rate. Write a Python program to calculate and display the converted amount.

**REFERENCES**


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no. : 4 | Date: |
|---|---|
| **Decision Control Structures** | |

## PROBLEM STATEMENTS

1. Write a Python program to input the gender (male/female), bus charge, and age of a passenger. If the passenger is a female or is older than 60 years, the ticket should be provided free of cost by setting the bus charge to 0. Display the charge.
2. Write a Python program to input a year from the user and determine whether it is a leap year or not. A year is a leap year if it is divisible by 400, or if it is divisible by 4 but not divisible by 100. Display an appropriate message indicating whether the given year is a leap year or a common year.
3. Write a Python program to input the number of electricity units consumed by a customer and calculate the electricity bill. If the consumption is up to 100 units, charge Rs. 5 per unit; if it is more than 100 and up to 300 units, charge Rs. 8 per unit; otherwise, charge Rs. 10 per unit. Display the total electricity bill amount.

## PRACTICAL SIGNIFICANCE

*This practice helps students understand how decision-making statements are used to solve real-world problems by evaluating conditions and executing appropriate actions.Students can learn to implement if, if-else, and if-elif-else statements effectively. These concepts form the foundation for developing intelligent programs that can make decisions based on user input and varying situations.*

## RELEVANT PROGRAM OUTCOMES(POs)

**PO1** – Apply knowledge of mathematics, natural sciences, engineering fundamentals, and an engineering specialization to practical procedures and practices.

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

## RELEVANT COURSE OUTCOMES(COs)

CO2: Optimize program execution through the strategic use of control structures and loop control statements

## THEORY

Decision-making statements in Python are used to control the flow of program execution based on conditions. These statements enable a program to evaluate an expression and execute different blocks of code depending on whether the condition is True or False. Decision making is an essential part of programming because it allows programs to react differently to different inputs and situations. It is widely used in applications such as billing systems, admission processing, eligibility verification, grading systems, and many other real-world scenarios.

Python provides several decision-making statements such as if, if-else, nested if, and if-elif-else.

**1. if Statement :** The if statement is the simplest decision-making statement. It executes a block of code only when the specified condition is true.

**Syntax:**

```
if condition:
    statement(s)
```

**2. if-else Statement :** The if-else statement is used when there are two possible outcomes. If the condition is true, the if block is executed; otherwise, the else block is executed.

**Syntax:**

```
if condition:
    statement(s)
else:
    statement(s)
```

**3. Nested if Statement**

A nested if statement contains one if statement inside another. It is used when a second condition needs to be checked only if the first condition is true.

**Syntax:**

```
if condition1:
    statement(s)
    if condition2:
        statement(s)
```

**4. if-elif-else Statement :** The if-elif-else statement is used to test multiple conditions sequentially. Once a condition evaluates to true, the corresponding block is executed and the remaining conditions are skipped.

**Syntax:**

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
elif condition3:
    statement(s)
else:
    statement(s)
```

These decision-making statements help programmers implement logical choices and control program execution efficiently. Understanding and using these statements is fundamental to developing interactive and intelligent Python applications.

## ALGORITHM/FLOWCHART

**Problem 1:**

1. Start.
2. Input gender, age, and bus charge.
3. Check if gender is female or age is greater than 60.
4. If the condition is true, set the bus charge to 0.
5. Display the bus charge.
6. Stop.

**Problem 2:**

1. Start.
2. Input a year.

3. Check whether the year is divisible by 4.
4. If yes, check whether it is divisible by 100.
5. If yes, check whether it is divisible by 400.
6. If divisible by 400, display "Leap Year"; otherwise, display "Not a Leap Year".
7. If not divisible by 100, display "Leap Year".
8. If not divisible by 4, display "Not a Leap Year".
9. Stop.

**Problem 3:**

1. Start.
2. Input the number of electricity units consumed.
3. If units are less than or equal to 100, calculate bills as units × 5.
4. Else if units are less than or equal to 300, calculate the bill as units × 8.
5. Otherwise, calculate the bill as units × 10.
6. Display the total bill amount.
7. Stop.

**PROGRAM CODE**

**Problem 1: Bus Ticket Generation**

```
gender = input("Enter gender (M/F): ")
age = int(input("Enter age: "))
charge = int(input("Enter bus charge: "))
if gender == "F" or age > 60:
          charge = 0
print("Bus Charge =", charge)
```

**Problem 2: Leap Year or Not**

```
year = int(input("Enter a year: "))
if year % 4 == 0:
    if year % 100 == 0:
        if year % 400 == 0:
            print("Leap Year")
        else:
            print("Not a Leap Year")
    else:
```

```
        print("Leap Year")
else:
    print("Not a Leap Year")
```

**Problem 3: Electricity Bill**

```
units = int(input("Enter the number of units consumed: "))
if units <= 100:
          bill = units * 5
elif units <= 300:
          bill = units * 8
else:
          bill = units * 10
print("Total Electricity Bill = Rs.", bill)
```

## PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.

```
python <filename>.py
```

6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

## RESULTS

The Python programs were executed successfully, and the required outputs were obtained accurately for the given inputs.

## CONCLUSION

Decision-making statements are used in Python to control the flow of a program based on conditions. They allow the program to choose different actions depending on whether a condition is true or false. In this experiment, various problems were

solved using if, if-else, and if-elif-else statements. The experiment helped in understanding conditional logic and its practical applications. It also improved problem-solving skills in Python programming.

## PRACTICAL RELATED QUESTIONS

1. ***What is a conditional statement in Python?***
   *A conditional statement is used to make decisions based on whether a condition evaluates to True or False.*
2. ***What is the purpose of indentation in Python decision-making statements?***
   *Indentation defines the block of code that belongs to a particular condition. Incorrect indentation can lead to errors or unexpected behavior.*
3. ***What is the difference between nested if and if-elif-else statements?***
   *Nested if statements place one if inside another, while if-elif-else is used to check multiple conditions in a structured sequence.*
4. ***Which operators are commonly used in decision-making statements?***
   *Comparison operators (==, !=, >, <, >=, <=) and logical operators (and, or, not) are commonly used.*
5. ***What happens if none of the conditions in an if-elif-else structure are true and there is no else block?***
   *The program simply skips the entire conditional structure and continues with the next statement.*

## SELF- LEARNING COMPONENTS

1. Prepare a self-study sheet containing definitions, flowcharts, syntax, and real-life examples of if and if- else statements
2. Write Python programs in online Python Compilers (onlineGDB/Programiz/Nextleap etc)

   ○ Write simple programs using if -else statements

## ADDITIONAL PROBLEM STATEMENTS

1. Write a Python program to input the age of a customer and calculate the movie ticket charge. If the age is below 12 years, charge Rs. 100; if the age is between 12 and 60 years, charge Rs. 200; otherwise, charge Rs. 150. Display the ticket amount.

2. Write a Python program to input the marks obtained by a student out of 100 and determine the grade. Assign Grade A for marks 90 and above, Grade B for marks between 75 and 89, Grade C for marks between 60 and 74, Grade D for marks between 40 and 59, and Grade F for marks below 40. Display the grade.
3. Write a Python program to input the monthly income and age of a person and determine whether the person is eligible for a loan. A person is eligible if the monthly income is above Rs. 25,000 and the age is between 21 and 60 years. Display an appropriate eligibility message.

**REFERENCES**


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no.: 5 | Date: |
|---|---|
| **Selection Based control structure** | |

## PROBLEM STATEMENTS

1. A school wants to automate the process of assigning grades to students based on their examination marks. The system should accept a student's mark and display the appropriate grade according to the school's grading policy.
2. A digital calendar application needs to display the day name when a user enters a day number (1–7). The program should match the entered number with the corresponding day.

## PRACTICAL SIGNIFICANCE

*Selection-based control structure statements, such as if-else ladder and match-case are widely used in real-world software applications. They enable programs to make logical decisions based on user inputs and specific conditions. These constructs are essential in systems such as banking applications, educational software, billing systems, reservation systems, and menu-driven applications, making programs more interactive, efficient, and user-friendly.*

## RELEVANT PROGRAM OUTCOMES(POs)

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

**PO9** – Communicate effectively and inclusively on well-defined engineering activities by comprehending the work of others and documenting their own work.

## RELEVANT COURSE OUTCOMES (COs)

**CO2** - Optimize program execution through the strategic use of control structures and loop control statements.

## THEORY

Selection-based control structure is an important feature of programming that allows a program to execute different sets of instructions based on certain conditions. In Python, decision-making can be performed using the if-else ladder and the match-case statement. These control structures help programmers develop applications that can make choices and respond differently to various inputs.

An if-else ladder is used when multiple conditions need to be evaluated. The program checks each condition sequentially from top to bottom. When a condition is found to be true, the corresponding block of code is executed, and the remaining conditions are ignored. If none of the conditions are true, the statements inside the else block are executed. This structure is commonly used in applications such as grading systems, employee salary classification, eligibility checking, and other situations involving range-based decisions.

**Syntax of If-Else Ladder:**

```
if condition1:
        statements
elif condition2:
        statements
elif condition3:
        statements
else:
        statements
```

Python 3.10 introduced the match-case statement, which serves a purpose similar to the Select Case statement available in many other programming languages. The match-case statement compares a given expression with multiple fixed values and executes the matching block of code. It makes programs more readable and easier to maintain when there are many possible choices based on a single variable. This structure is widely used in menu-driven programs, day-name selection, month-name display, and command-processing applications.

**Syntax of Match-Case (Select Case):**

```
match expression:
          case value1:
                statements
          case value2:
                statements
          case value3:
                statements
          case _:
                statements
```

Here, the underscore (_) acts as the default case and is executed when none of the specified cases match the expression.

Both if-else ladder and match-case are powerful decision-making tools in Python. The if-else ladder is more suitable for checking complex conditions and value ranges, while match-case is ideal for selecting one action from multiple fixed options. Proper use of these structures improves program efficiency, readability, and maintainability.

## ALGORITHM/FLOWCHART

### 1. Student Grade Evaluation

1. Start.
2. Input the student's mark.
3. If the mark is greater than or equal to 90, display Grade A.
4. Else if the mark is greater than or equal to 80, display Grade B.
5. Else if the mark is greater than or equal to 70, display Grade C.
6. Else if the mark is greater than or equal to 60, display Grade D.
7. Otherwise, display Fail.
8. Stop

### 2. Day Name Display Start.

1. Input the day number (1–7).
2. Match the day number with the corresponding day.
3. If the number is:
    - 1 → Sunday
    - 2 → Monday

- 3 → Tuesday
- 4 → Wednesday
- 5 → Thursday
- 6 → Friday
- 7 → Saturday

4. If the number is not between 1 and 7, display Invalid Day Number.
5. Stop.

**PROGRAM CODE**

**Problem 1 :**

```
mark = int(input("Enter the student's mark: "))
if mark >= 90:
        print("Grade A")
elif mark >= 80:
        print("Grade B")
elif mark >= 70:
        print("Grade C")
elif mark >= 60:
        print("Grade D")
else:
        print("Fail")
```

**Problem 2 :**

```
day = int(input("Enter day number (1-7): "))
match day:
        case 1:
              print("Sunday")
        case 2:
              print("Monday")
        case 3:
              print("Tuesday")
        case 4:
              print("Wednesday")
        case 5:
              print("Thursday")
        case 6:
              print("Friday")
```

```
        case 7:
              print("Saturday")
        case _:
              print("Invalid Day Number")
```

**PROCEDURE**

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as <filename>.py.
5. Run the program.
   python <filename>.py
6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

**RESULTS**

Python programs using if-else ladder and match-case statements were successfully designed, implemented, and executed. The desired outputs were obtained for different inputs, demonstrating the effective use of decision-making constructs in Python.

**CONCLUSION**

Python program was successfully developed using the match-case statement to display the corresponding day name for a given day number. The program demonstrated how match-case can be used to simplify multiple-choice selection problems and improve code readability

**PRACTICAL RELATED QUESTIONS**

1. ***What is the purpose of the elif keyword?***

*The elif keyword is used to check additional conditions if the previous conditions are false.*

2. ***What is the default block in an if-else ladder?***

*The else block acts as the default block and executes when all conditions are false.*

3. ***What is the Python equivalent of Select Case?***

*The match-case statement introduced in Python 3.10 is the equivalent of Select Case.*

***4. What is the purpose of the match statement?***

*The match statement compares an expression with multiple values and executes the matching case.*

***5. What does the underscore (_) represent in match-case?***

*It represents the default case and executes when no other case matches.*

## SELF- LEARNING COMPONENTS

- Draw a flowchart of match-case
- Compare the match-case statement with the if-elif ladder by analyzing their advantages and limitations through examples
- Identify and list problems suitable for solving with the match–case structure

## ADDITIONAL PROBLEM STATEMENTS

1. A calendar system should display the month name when a user enters a month number (1–12). The program should match the number with the appropriate month.
2. A bank wants to check whether a customer is eligible for a loan based on age, monthly income, and credit score. The system should display the eligibility status.

## REFERENCES


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no.: 6 | Date: |
|---|---|
| **Loop Control Structures** | |

## PROBLEM STATEMENTS

1. A student wants to learn multiplication tables. Write a Python program to generate the multiplication table of a given number using a loop.
2. A website requires users to enter the correct password before accessing their account. Write a Python program using a while loop that repeatedly asks for the password until the correct one is entered.
3. A bank wants to verify account numbers by calculating the sum of all digits in a customer's account number. Write a Python program to accept a number from the user and find the sum of its digits using a loop.
4. A school administrator wants to count the number of students entering a classroom until the expected strength is reached. Write a Python program using a while loop to perform the counting

## PRACTICAL SIGNIFICANCE

*Loop control structures are widely used in real-world applications such as processing student records, generating reports, validating user input, searching data, and automating repetitive tasks. They improve program efficiency by allowing the same set of instructions to be executed multiple times without rewriting code*

## RELEVANT PROGRAM OUTCOMES(POs)

**PO1** – Apply knowledge of mathematics, natural sciences, engineering fundamentals, and an engineering specialization to practical procedures and practices.

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

## RELEVANT COURSE OUTCOMES (COs)

**CO2** - Optimize program execution through the strategic use of control structures and loop control statements.

## THEORY

Loop control structures in Python are used to execute a block of code repeatedly until a specified condition is met. They help automate repetitive tasks, reduce code duplication, and improve program efficiency. Python provides two main looping constructs: **for loop** and **while loop**. Additionally, loop control statements such as **break**, **continue**, and **pass** can alter the normal flow of loop execution.

**1. for Loop :** A for loop is used to iterate over a sequence such as a list, tuple, string, or range of numbers.

**Syntax:**

```
for variable in sequence:
        statement(s)
```

**2. while Loop :** A while loop executes as long as a specified condition remains true.

**Syntax:**

```
while condition:
        statement(s)
```

The break statement terminates the loop immediately and transfers control to the statement following the loop.The continue statement skips the current iteration and proceeds with the next iteration of the loop.The pass statement is a null statement. It does nothing and is used as a placeholder where code will be added later.

The **range()** function in Python is a built-in function used to generate a sequence of numbers. It is commonly used with **for loops** to control the number of iterations. The range() function does not store all numbers in memory at once; instead, it generates them as needed, making it memory efficient.

The sequence generated by range() starts from a specified value, increases or decreases by a specified step value, and stops before reaching the ending value.

**Syntax:**

```
range(start, stop, step)
```

**ALGORITHM/FLOWCHART**

**Problem 1: Multiplication Table of a Given Number**

1. Start.
2. Read a number n.
3. Use a loop from 1 to 10.
4. Multiply n by the loop variable.
5. Display the result.
6. Stop

**Problem 2: Password Verification Using While Loop**

1. Start.
2. Set the correct password.
3. Ask the user to enter a password.
4. If the password is incorrect, ask again.
5. Repeat until the correct password is entered.
6. Display "Access Granted".
7. Stop.

**Problem 3: Sum of Digits of a Number**

1. Start.
2. Read a number n.
3. Initialize sum = 0.
4. While n > 0:
   - Find the last digit using n % 10.
   - Add the digit to sum.
   - Remove the last digit using n // 10.
5. Display the sum.
6. Stop.

**Problem 4: Counting Students Entering a Classroom**

1. Start.
2. Read the expected classroom strength.
3. Initialize count = 0.

4. While count is less than the expected strength:
   - Increase count by 1.
   - Display the current count.
5. When the expected strength is reached, display a message.
6. Stop.

**PROGRAM CODE**

**Problem 1: Multiplication Table of a Given Number**

```
n = int(input("Enter a number: "))

for i in range(1, 11):

          print(n, "x", i, "=", n * i)
```

**Problem 2: Password Verification Using While Loop**

```
correct_password = "python123"

password = input("Enter password: ")

while password != correct_password:

          print("Incorrect Password!")

          password = input("Enter password again: ")

print("Access Granted")
```

**Problem 3: Sum of Digits of a Number**

```
n = int(input("Enter a number: "))

sum_digits = 0

while n > 0:
```

```
        digit = n % 10
        sum_digits += digit
        n = n // 10
print("Sum of digits =", sum_digits)
```

**Problem 4: Counting Students Entering a Classroom**

```
strength = int(input("Enter expected classroom strength: "))
count = 0
while count < strength:
        count += 1
        print("Student", count, "entered the classroom")
print("All students have entered the classroom.")
```

## PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.

```
python <filename>.py
```

6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result..

## RESULTS

The Python programs were successfully implemented using **for loops** and **while loops** .

## CONCLUSION

The experiment was successfully completed by implementing Python programs using loop control structures such as **for** and **while** loops. The programs demonstrated how loops can be used to perform repetitive tasks efficiently, including generating multiplication tables, validating passwords, calculating the sum of digits, and counting students. This experiment helped in understanding the importance of loops in automating real-world processes and improving program efficiency.

## PRACTICAL RELATED QUESTIONS

1. ***What are the types of loops available in Python?***

   *Python provides two types of loops: for loop and while loop*

2. ***What is the purpose of the range() function?***

   *The range() function generates a sequence of numbers, commonly used with for loops.*

3. ***What is an infinite loop?***

   *A loop that continues indefinitely because its condition never becomes false.*

4. ***Can loops work with strings in Python?***

   *Yes, a for loop can iterate through each character of a string.*

5. ***What is the use of the break statement?***

   *The break statement immediately terminates a loop.*

## SELF- LEARNING COMPONENTS

1. Explain the situations in which break, continue, and pass statements are used, with examples.
2. Prepare a short comparison chart of For Loop and While Loop.
3. Identify and debug infinite loop situations and document methods to prevent them

## ADDITIONAL PROBLEM STATEMENTS

1. A rocket launch system displays a countdown before launch. Write a Python program using a loop to display numbers from 10 to 1.
2. A language learning application wants to count the number of vowels in a word entered by the user. Write a Python program using a loop to count vowels.

3. A school wants to identify whether roll numbers from 1 to N are odd or even. Write a Python program using a loop to display odd and even roll numbers.

**REFERENCES**


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no.: 7 | Date: |
|---|---|
| **String Manipulations** | |

## PROBLEM STATEMENTS

An e-commerce company wants to develop a Customer Feedback and Account Management System that processes customer details and feedback using Python string operations.

1. Accept the customer's name, customer ID, email address, and feedback message.
2. Display the first and last characters of the customer's name using indexing.
3. Extract the first five characters and last five characters of the feedback using slicing.
4. Concatenate the customer's first name and customer ID to create a unique username.
5. Check whether the word "good" is present in the feedback using membership operators.
6. Convert the feedback into:
   - Uppercase for report generation
   - Lowercase for text analysis
   - Capitalized format for display
7. Replace the word "bad" with "improve" in the feedback.
8. Find the position of the word "delivery" in the feedback.
9. Count the occurrences of the word "good" in the feedback.
10. Remove extra spaces from the customer's input using strip().
11. Split the feedback into individual words using split().
12. Check whether the email address contains the symbol @ using membership operators.

## PRACTICAL SIGNIFICANCE

*This experiment helps students understand the use of Python string operations in real-world applications such as customer management and feedback processing systems. Students learn to manipulate, validate, search, and format textual data using techniques like indexing, slicing, concatenation, membership testing, and string methods. These concepts are widely*

*used in e-commerce platforms, registration systems, email validation, and text-processing applications, helping students develop practical programming and problem-solving skills.*

**RELEVANT PROGRAM OUTCOMES(POs)**

**PO2 –** Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3 –** Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5 –** Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

**PO9 –** Communicate effectively and inclusively on well-defined engineering activities.

**RELEVANT COURSE OUTCOMES (COs)**

**CO3** - Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations, built-in methods to solve real-world programming problems.

**THEORY**

A **string** is a sequence of characters enclosed in single quotes (' '), double quotes (" "), or triple quotes (''' ''' / """ """). Strings in Python are immutable; any modification operation will always return a new string rather than altering the original one in place.

1. **Creating a String** : Strings can be created using either single ('...') or double ("...") quotes. Both behave the same.Use triple quotes ('''...''' ) or ( """...""") for strings that span multiple lines. They preserve line breaks and formatting exactly as written.
2. **Indexing** : Indexing is the process of accessing individual characters in a string using their position (index number). In Python, each character in a string is assigned an index starting from **0**.Positive indices start at 0 from the left, negative indices start at -1 from the right
3. **String Slicing** - String Slicing is the process of extracting a portion (substring) of a string using index positions. Python uses the slicing operator [:] to access a range of characters from a string.

   **Syntax:**

```
string[start : stop : step]
```

4. **String Operations** : String Operations are the various actions that can be performed on strings.
   - Concatenation (+) : Concatenation combines two or more strings into a single string.
   - Repetition (*) :Repetition duplicates a string a specified number of times.
   - Membership Operators (in, not in) : Checks whether a substring exists in a string.
5. **String Methods** : String methods are built-in functions provided by Python that perform various operations on strings

| Method | Description |
|---|---|
| upper() | Converts all characters to uppercase |
| lower() | Converts all characters to lowercase |
| capitalize() | Converts the first character to uppercase |
| replace(old, new) | Replaces a substring with another substring |
| find(sub) | Returns the index of the first occurrence of a substring |
| count(sub) | Counts the occurrences of a substring |
| strip() | Removes spaces from both ends |
| split(sep) | Splits a string into a list |
| isdigit() | Returns True if all characters are digits |

## ALGORITHM/FLOWCHART

**Problem 1:**

1. Start
2. Input the customer's name, customer ID, email address, and feedback message.
3. Remove extra spaces from the customer's name, email, and feedback using strip().
4. Display the first and last characters of the customer's name using indexing.
5. Extract the first five and last five characters of the feedback using slicing.

6. Split the customer's name and extract the first name.
7. Concatenate the first name and customer ID to create a unique username.
8. Check whether the word "good" is present in the feedback using the membership operator in.
9. Convert the feedback to uppercase using upper().
10. Convert the feedback to lowercase using lower().
11. Convert the feedback to capitalized format using capitalize().
12. Replace the word "bad" with "improve" using replace().
13. Find the position of the word "delivery" using find().
14. Count the occurrences of the word "good" using count().
15. Split the feedback into individual words using split().
16. Check whether the email contains the symbol @ using the membership operator.
17. Display all results.
18. Stop.

**PROGRAM CODE**

**Problem 1:**

```
# Customer Feedback and Account Management System
# Input customer details
name = input("Enter Customer Name: ").strip()
customer_id = input("Enter Customer ID: ").strip()
email = input("Enter Email Address: ").strip()
feedback = input("Enter Feedback: ").strip()
# Indexing
first_char = name[0]
last_char = name[-1]
# Slicing
first_five = feedback[:5]
last_five = feedback[-5:]
# Concatenation
first_name = name.split()[0]
username = first_name + customer_id
# Membership operator
good_present = "good" in feedback.lower()
# Case conversion
upper_feedback = feedback.upper()
lower_feedback = feedback.lower()
```

```
capitalized_feedback = feedback.capitalize()
# Replace operation
modified_feedback = feedback.replace("bad", "improve")
# Find operation
delivery_position = feedback.lower().find("delivery")
# Count operation
good_count = feedback.lower().count("good")
# Split operation
feedback_words = feedback.split()
# Email validation
email_valid = "@" in email
# Display Results
print("\n----- Customer Feedback Analysis -----")
print("Customer Name :", name)
print("Customer ID :", customer_id)
print("Email :", email)
print("\nFirst Character of Name :", first_char)
print("Last Character of Name :", last_char)
print("\nFirst 5 Characters of Feedback :", first_five)
print("Last 5 Characters of Feedback :", last_five)
print("\nUsername :", username)
print("\nIs 'good' present in feedback? :", good_present)
print(“\n”)
print(“=”*20)
print("\nFeedback in Uppercase :")
print(upper_feedback)
print("\nFeedback in Lowercase :")
print(lower_feedback)
print("\nFeedback in Capitalized Form :")
print(capitalized_feedback)
print("\nFeedback after replacing 'bad' with 'improve' :")
print(modified_feedback)
print("\nPosition of 'delivery' :", delivery_position)
print("\nOccurrences of 'good' :", good_count)
print("\nFeedback split into words :")
print(feedback_words)
if email_valid:
    print("\nValid Email Address")
```

```
else:
    print("\nInvalid Email Address")
```

**PROCEDURE**

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.

```
python <filename>.py
```

6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

**RESULTS**

The Python program to perform string manipulations was successfully implemented and executed.

**CONCLUSION**

The program demonstrated various string concepts such as indexing, slicing, concatenation, membership operators, and built-in string methods including upper(), lower(), capitalize(), replace(), find(), count(), strip(), and split(). These operations are essential for processing, validating, and analyzing textual data in real-world applications

**PRACTICAL RELATED QUESTIONS**

1. ***How are strings represented in python?***
   - *Single quotes: 'Hello'*
   - *Double quotes: "Hello"*
   - *Triple quotes: '''Hello''' or """Hello"""*
2. ***Are strings mutable in Python?***
   *No, strings are immutable. String methods return a new string without changing the original string.*
3. ***What is negative indexing?***
   *Negative indexing accesses characters from the end of the string.*
4. ***What is the purpose of the upper() method?***
   *It converts all characters in a string to uppercase*

5. ***What is the use of the count() method?***
   *Counts the number of occurrences of a substring.*

## SELF- LEARNING COMPONENTS

1. Create documentation outlining common string methods and their usage with examples
2. Perform string slicing operations with positive and negative indexing

## ADDITIONAL PROBLEM STATEMENTS

A food delivery company wants to analyze customer reviews.

1. Accept customer name, order ID, email address, and review message.
2. Display the first and last characters of the customer's name.
3. Extract the first and last five characters of the review.
4. Create a username using the customer's first name and order ID.
5. Check whether the word "tasty" exists in the review.
6. Convert the review into uppercase, lowercase, and capitalized forms.
7. Replace the word "late" with "delayed".
8. Find the position of the word "delivery".
9. Count the occurrences of the word "good".
10. Remove unnecessary spaces.
11. Split the review into words.
12. Validate the email address using membership operators.

## REFERENCES


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |

| Experiment no. : 8 | Date: |
|---|---|
| **List Operations** | |

**PROBLEM STATEMENTS**

A supermarket wants to maintain a list of products available in its inventory. Develop a Python program using lists to store and manage product names and perform various list operations.

1. Create a list of product names.
2. Display the complete list of products.
3. Access and display the first and last product using indexing.
4. Add a new product to the inventory using append().
5. Insert a product at a specified position using insert().
6. Remove a product from the inventory using remove().
7. Display the total number of products in the list using len().
8. Check whether a particular product exists in the inventory using membership operators (in and not in).
9. Display products using list slicing.
10. Sort the products in alphabetical order.
11. Reverse the order of products.
12. Count the occurrence of a specified product in the list.
13. Find the index position of a particular product.
14. Display the final updated inventory list.

**PRACTICAL SIGNIFICANCE**

*Lists are widely used in real-world applications to manage collections of related data. Understanding list operations helps programmers efficiently organize, access, modify, and analyze sequential data in software applications.*

**RELEVANT PROGRAM OUTCOMES (POs)**

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

**PO10** – Demonstrate awareness of engineering management principles and apply them to manage projects in multidisciplinary environments.

**RELEVANT COURSE OUTCOMES(COs)**

**CO3** - Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations, built-in methods to solve real-world programming problems

**THEORY**

A **list** is one of the most versatile and widely used data structures in Python. It is an ordered and mutable collection that allows multiple items to be stored in a single variable. Lists are used to manage sequential data such as student marks, employee records, product inventories, customer details, and other collections of related information. A list is created using square brackets [], with elements separated by commas. Since lists are ordered, each element has a specific position called an index, which starts from 0. Python also supports negative indexing, allowing elements to be accessed from the end of the list.

Lists support various operations that make data manipulation easy and efficient. **Element access** allows individual items to be retrieved using their index values. **Concatenation** combines two or more lists into a single list using the + operator, while **repetition** creates multiple copies of a list using the * operator. Python also provides **membership operators** such as in and not in to check whether a particular element exists within a list.

| Method | Description |
|---|---|
| append() | Adds an element to the end of the list |
| insert() | Inserts an element at a specified position |
| remove() | Removes a specified element |

| | |
|---|---|
| pop() | Removes an element at a given index |
| sort() | Sorts the list |
| reverse() | Reverses the list |
| count() | Returns the number of occurrences |
| index() | Returns the index of an element |

## ALGORITHM/FLOWCHART

1. Start.
2. Create a list of product names.
3. Display the complete list of products.
4. Display the first product using index 0.
5. Display the last product using index -1.
6. Input a new product and add it to the list using append().
7. Input a position and product name, then insert the product using insert().
8. Input a product name to remove and delete it using remove().
9. Find and display the total number of products using len().
10. Input a product name to search.
11. Check whether the product exists using the in operator.
12. Display a portion of the list using slicing.
13. Sort the list alphabetically using sort().
14. Reverse the list using reverse().
15. Input a product name and count its occurrences using count().
16. Input a product name and find its index position using index().
17. Display the final updated inventory list.
18. Stop.

## PROGRAM CODE

```
# Product Inventory Management using Lists
# Create a list of product names
products = ["Laptop", "Mouse", "Keyboard", "Monitor", "Printer"]
```

```
# Display the complete list
for product in products:
          print(products)
# Access first and last product
print("\nFirst Product:", products[0])
print("Last Product:", products[-1])
# Add a new product
new_product = input("\nEnter a product to add: ")
products.append(new_product)
print("After Append:", products)
# Insert a product at a specified position
position = int(input("\nEnter position to insert product: "))
product = input("Enter product name: ")
products.insert(position, product)
print("After Insert:", products)
# Remove a product
remove_product = input("\nEnter product to remove: ")
if remove_product in products:
    products.remove(remove_product)
    print("After Remove:", products)
else:
    print("Product not found.")
# Display total number of products
print("\nTotal Number of Products:", len(products))
# Membership testing
search_product = input("\nEnter product to search: ")
if search_product in products:
    print(search_product, "is available in inventory.")
else:
    print(search_product, "is not available in inventory.")
# Display products using slicing
print("\nFirst Three Products:", products[:3])
print("Last Three Products:", products[-3:])
# Sort products alphabetically
products.sort()
print("\nSorted Product List:")
print(products)
# Reverse the order
```

```
products.reverse()
print("\nReversed Product List:")
print(products)
# Count occurrence of a product
count_product = input("\nEnter product to count: ")
print("Occurrences:", products.count(count_product))
# Find index position
index_product = input("\nEnter product to find index: ")
if index_product in products:
    print("Index Position:", products.index(index_product))
else:
    print("Product not found in the list.")
# Display final updated inventory
print("\nFinal Updated Inventory:")
print(products)
```

## PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.

   ```
   python <filename>.py
   ```
6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

## RESULTS

The Python program to perform List operations such as creating, appending, membership testing,removing,sorting,finding items and reversing was successfully implemented and executed.

## CONCLUSION

Learned how to create and manipulate lists in Python and perform different list operations for managing sequential data efficiently. Lists provide a flexible and powerful way to store, access, update, and organize collections of related data in real-world applications.

## PRACTICAL RELATED QUESTIONS

1. ***How do you create a list in Python?***

   *A list is created using square brackets.*

2. ***How do you add an element to the end of a list?***

   *By using the append() method.*

3. ***What does the len() function do?***

   *It returns the total number of elements in a list.*

4. ***What does the count() method do?***

   *It returns the number of times a specified element appears in a list.*

5. ***Can a list contain different data types?***

   *Yes. A list can store integers, strings, floats, and other objects together.*

## SELF- LEARNING COMPONENTS

1. Prepare a Tutorial Video Script or Video about Python List (5 minutes)
2. Document the different operations on lists along with relevant built-in functions

## ADDITIONAL PROBLEM STATEMENTS

1. Create a list of book titles and perform the following:
   - Display all books.
   - Add new books.
   - Remove damaged books.
   - Search for a particular book.
   - Count the number of books.
   - Display books in alphabetical order.

2. Create a list of cricket players and:
   - Display all players.
   - Add a new player.
   - Remove an injured player.
   - Search for a player.
   - Count the total number of players.
   - Display players in sorted order.

**REFERENCES**


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no. : 9 | Date: |
|---|---|
| **Dictionary Operations** | |

## PROBLEM STATEMENTS

A supermarket wants to maintain product information using a dictionary. Develop a Python program to store product IDs as keys and product names as values and perform various dictionary operations.

1. Create a dictionary containing product IDs and product names.
2. Display the complete dictionary.
3. Access and display a product using its key.
4. Add a new product to the dictionary.
5. Update an existing product name.
6. Remove a product using pop().
7. Display all product IDs using keys().
8. Display all product names using values().
9. Display all key-value pairs using items().
10. Check whether a particular product ID exists using membership operators.
11. Find the total number of products using len().
12. Display the final updated dictionary.

## PRACTICAL SIGNIFICANCE

*Dictionaries are widely used in real-world applications to store and manage data in the form of key-value pairs. They provide fast and efficient access, updating, and retrieval of information using unique keys. Understanding dictionary operations helps in developing applications such as inventory management systems, student databases, and employee record management systems.*

## RELEVANT PROGRAM OUTCOMES (POs)

**PO2** - Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** - Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

**PO10** – Demonstrate awareness of engineering management principles and apply them to manage projects in multidisciplinary environments.

**RELEVANT COURSE OUTCOMES (COs)**

**CO3** - Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations, built-in methods to solve real-world programming problems

**THEORY**

A **dictionary** is a built-in data structure in Python used to store data in the form of **key-value pairs**. Each key in a dictionary is unique and is used to access its corresponding value. Dictionaries are enclosed within curly braces {} and are separated by commas. Unlike lists, dictionaries do not use numerical indexes for accessing data; instead, values are retrieved using their associated keys. Dictionaries are mutable, which means elements can be added, modified, or removed after creation. They are widely used in real-world applications such as student databases, employee records, product inventories, contact management systems, and configuration settings where data needs to be organized and accessed efficiently using meaningful identifiers.

Python provides several operations and methods for working with dictionaries. Values can be accessed using keys, new key-value pairs can be added, and existing values can be updated.

| Method | Description |
|---|---|
| get() | Returns the value associated with the specified key. |
| keys() | Returns all keys in the dictionary. |
| values() | Returns all values in the dictionary. |
| items() | Returns all key-value pairs in the dictionary as tuples. |

| | |
|---|---|
| pop() | Removes the specified key and returns its value. |
| popitem() | Removes and returns the last inserted key-value pair. |

**ALGORITHM/FLOWCHART**

1. Start.
2. Create a dictionary containing product IDs and product names.
3. Display the dictionary.
4. Access a product using its key.
5. Add a new key-value pair.
6. Update an existing value.
7. Remove a product using pop().
8. Display keys using keys().
9. Display values using values().
10. Display key-value pairs using items().
11. Check whether a key exists using in.
12. Display the number of items using len().
13. Display the final dictionary.
14. Stop.

**PROGRAM CODE**

```
# Dictionary Operations
products = {
    101: "Laptop",
    102: "Mouse",
    103: "Keyboard",
    104: "Monitor"
}
# Display dictionary
print("Product Dictionary:")
print(products)
# Access value
print("\nProduct with ID 102:", products[102])
# Add new product
products[105] = "Printer"
print("\nAfter Adding Product:")
print(products)
```

```
# Update product
products[103] = "Mechanical Keyboard"
print("\nAfter Updating Product:")
print(products)
# Remove product
products.pop(104)
print("\nAfter Removing Product:")
print(products)
# Display keys
print("\nProduct IDs:")
print(products.keys())
# Display values
print("\nProduct Names:")
print(products.values())
# Display items
print("\nKey-Value Pairs:")
print(products.items())
# Membership testing
if 102 in products:
    print("\nProduct ID 102 Exists")
else:
    print("\nProduct ID 102 Not Found")
# Length
print("\nTotal Products:", len(products))
# Final dictionary
print("\nFinal Dictionary:")
print(products)
```

**PROCEDURE**

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.

   ```
   python <filename>.py
   ```
6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.

8. Repeat the testing process with different input values.
9. Record the output and result.

**RESULTS**

The dictionary was successfully created and manipulated using various dictionary operations such as adding, updating, deleting, accessing, and searching elements. The final updated dictionary was displayed correctly.

**CONCLUSION**

The experiment was completed successfully. We learned how to create and manipulate dictionaries in Python and perform operations such as insertion, deletion, updating, searching, and displaying key-value pairs. Dictionaries provide an efficient way to store and manage structured data using unique keys

**PRACTICAL RELATED QUESTIONS**

1. ***How is a dictionary created?***

   *Using curly braces {}.*

2. ***What is a key-value pair?***

   *A key is used to identify and access its associated value.*

3. ***Are dictionary keys unique?***

   *Yes, dictionary keys must be unique.*

4. ***Are dictionaries mutable?***

   *Yes, dictionaries can be modified after creation.*

5. ***Can dictionary values be duplicated?***

   *Yes, values can be duplicated.*

**SELF- LEARNING COMPONENTS**

1. Prepare a self-study sheet containing usage, operations and built-in methods of Dictionary

2. Create a Python program to manage student records using a dictionary. Your program should:
   - Create a dictionary containing at least 5 students and their marks.
   - Access the marks of a specific student, and safely handle the case where the student does not exist.
   - Add a new student to the dictionary.
   - Update the marks of an existing student

**ADDITIONAL PROBLEM STATEMENTS**

A school wants to maintain student details using a dictionary where the roll number is the key and the student name is the value.

1. Create a dictionary of student records.
2. Display all student details.
3. Access a student using the roll number.
4. Add a new student record.
5. Update an existing student name.
6. Remove a student record.
7. Display all roll numbers using keys().
8. Display all student names using values().
9. Check whether a roll number exists.
10. Display the final updated dictionary.

**REFERENCES**


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no. : 10 | Date: |
|---|---|
| **Set Operations** | |

## PROBLEM STATEMENTS

A library wants to maintain a set of registered member names. Develop a Python program using sets to manage library memberships and perform various set operations.

1. Create a set of registered library members.
2. Display the complete set of members.
3. Add a new member using add().
4. Remove a member who has cancelled membership using remove().
5. Check whether a particular member is registered using membership operators (in and not in).
6. Display the total number of registered members using len().
7. Create another set containing members registered in a different library branch.
8. Find the union of the two member sets to obtain all registered members.
9. Find the intersection of the two sets to identify members registered in both branches.
10. Find the difference between the two sets to identify members unique to the first branch.
11. Display the final updated set of members.
12. Display all results of the set operations.

## PRACTICAL SIGNIFICANCE

*Sets are useful in real-world applications where duplicate data must be eliminated and efficient searching is required.Set operations like union, intersection, and difference are useful for comparing groups, identifying common members, and generating reports for decision-making and resource management.*

## RELEVANT PROGRAM OUTCOMES(POs)

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3 –** Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5 –** Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

**PO10 –** Demonstrate awareness of engineering management principles and apply them to manage projects in multidisciplinary environments.

**RELEVANT COURSE OUTCOMES(COs)**

**CO3** - Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations, built-in methods to solve real-world programming problems.

**THEORY**

A **set** is an unordered collection of unique elements in Python. Sets are used when duplicate values are not allowed and when fast membership testing is required. A set is created using curly braces {} or the set() function. Since sets are unordered, they do not support indexing or slicing. Sets are mutable, meaning elements can be added or removed after creation, but the individual elements must be immutable. Sets are commonly used in applications such as student registration systems, inventory management, data analysis, and removing duplicate values from collections.

Python provides various set operations such as union, intersection, difference, and symmetric difference, which are useful for comparing and combining collections of data.

| Method | Description |
|---|---|
| add() | Adds an element to the set. |
| remove() | Removes the specified element; raises an error if not fou |
| pop() | Removes and returns a random element from the set. |
| union() | Returns a new set containing elements from both sets. |

| | |
|---|---|
| intersection() | Returns common elements from two sets. |
| difference() | Returns elements present in the first set but not in the second. |

**ALGORITHM/FLOWCHART**

1. Start.
2. Create a set of registered library members.
3. Display the complete set of members.
4. Input a new member name and add it to the set using add().
5. Input a member name to remove and delete it using remove().
6. Input a member name to search.
7. Check whether the member exists using the in operator.
8. Display the total number of registered members using len().
9. Create another set containing members from a different library branch.
10. Display the second set.
11. Find and display the union of the two sets.
12. Find and display the intersection of the two sets.
13. Find and display the difference between the two sets.
14. Display the final updated set of members.
15. Stop.

**PROGRAM CODE**

```
# Library Member Registration System
# Create a set of registered members
members = {"Anu", "Rahul", "Meera", "John"}

# Display complete set
print("Library Members:")
print(members)

# Add a new member
new_member = input("\nEnter a new member name to add: ")
members.add(new_member)
print("After Adding Member:")
print(members)
```

```
# Remove a member
remove_member = input("\nEnter a member name to remove: ")
if remove_member in members:
    members.remove(remove_member)
    print("After Removing Member:")
    print(members)
else:
    print("Member not found.")

# Membership testing
search_member = input("\nEnter a member name to search: ")

if search_member in members:
    print(search_member, "is registered in the library.")
else:
    print(search_member, "is not registered in the library.")

# Display total members
print("\nTotal Number of Members:", len(members))

# Create another set of members
branch2 = {"Rahul", "Arun", "Neha", "Meera", "David"}

print("\nMembers in Branch 2:")
print(branch2)

# Union
print("\nUnion of Members:")
print(members.union(branch2))

# Intersection
print("\nCommon Members:")
print(members.intersection(branch2))

# Difference
print("\nMembers Only in Branch 1:")
print(members.difference(branch2))
```

```
# Final updated set
print("\nFinal Updated Member Set:")
print(members)
```

## PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.

```
python <filename>.py
```

6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

## RESULTS

The set was successfully created and manipulated using various set operations such as adding, removing, membership testing, union, intersection, difference, and symmetric difference. The results were displayed correctly.

## CONCLUSION

We learned how to create and manipulate sets in Python and perform different set operations. Sets provide an efficient way to store unique elements and perform mathematical set operations for data analysis and management.

## PRACTICAL RELATED QUESTIONS

### 1. *How is a set created?*

*Using curly braces {} or the set() function.*

### 2. *Are duplicate values allowed in a set?*

*No, duplicate values are automatically removed.*

### 3. *Are sets ordered?*

*No, sets are unordered.*

4. ***Can we use indexing in sets?***

   *No, sets do not support indexing*

5. ***What does intersection() do?***

   *Returns common elements from two sets*

**SELF- LEARNING COMPONENTS**

Create a Python program to

○ Create two sets with some common and some unique elements.

○ Perform operations:

- ■ Union
- ■ Intersection
- ■ Difference

**ADDITIONAL PROBLEM STATEMENTS**

A sports club wants to maintain a set of members participating in various sports activities. Develop a Python program using sets to manage member registrations and perform different set operations.

1. Create a set of sports club members.
2. Display the complete set of members.
3. Add a new member using add().
4. Remove a member who has left the club using remove().
5. Check whether a particular member is registered using membership operators (in and not in).
6. Display the total number of members using len().
7. Create another set containing members of a fitness club.
8. Find the union of the two sets to get all registered members.
9. Find the intersection of the two sets to identify members common to both clubs.
10. Find the difference between the two sets to identify members unique to the sports club.
11. Display the final updated set of sports club members.

## REFERENCES

| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no. : 11 | Date: |
|---|---|
| **Built in Functions in Python** | |

## PROBLEM STATEMENTS

A teacher wants to analyze the marks of students in a class. Develop a Python program that accepts a list of student marks and uses Python's built-in functions to perform various calculations and analyses.

1. Create a list of student marks.
2. Display the complete list of marks.
3. Find the total number of marks using len().
4. Find the highest mark using max().
5. Find the lowest mark using min().
6. Calculate the sum of all marks using sum().
7. Calculate the average mark.

## PRACTICAL SIGNIFICANCE

*Functions are widely used in software development to divide complex programs into smaller, manageable modules. Built-in functions save development time by providing ready-made operations, Functions are extensively used in banking systems, inventory management, payroll processing, data analysis, and web applications to improve code reusability and reduce redundancy.*

## RELEVANT PROGRAM OUTCOMES (POs)

**PO1** – Apply knowledge of mathematics, natural sciences, engineering fundamentals, and an engineering specialization to practical procedures and practices.

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

**RELEVANT COURSE OUTCOMES (COs)**

**CO4** - Design and develop modular programs using functions, modules, and packages with proper scope management and argument passing techniques.

**THEORY**

A **function** is a block of organized, reusable code that performs a specific task. Functions help reduce code repetition, improve readability, and make programs easier to maintain. Python provides two main types of functions: **Built-in Functions** and **User-Defined Functions**.

**Built-in functions** are pre-defined functions provided by Python that can be used directly without defining them. These functions perform common operations such as mathematical calculations, type conversions, input/output operations, and data processing.Built-in functions simplify programming by providing ready-made solutions for frequently used tasks.

| Function | Description |
|---|---|
| print() | Displays output on the screen. |
| input() | Accepts input from the user. |
| len() | Returns the length of an object. |
| type() | Returns the data type of an object. |
| max() | Returns the largest value. |
| min() | Returns the smallest value. |
| sum() | Returns the sum of elements. |

| | |
|---|---|
| abs() | Returns the absolute value. |
| round() | Rounds a number to the nearest value. |
| range() | Generates a sequence of numbers. |

**ALGORITHM/FLOWCHART**

1. Start.
2. Create a list of student marks.
3. Display the complete list of marks.
4. Find the total number of marks using len().
5. Find the highest mark using max().
6. Find the lowest mark using min().
7. Calculate the sum of all marks using sum().
8. Calculate the average mark using: Average=Sum of Marks\Number of Marks
9. Display the total number of marks, highest mark, lowest mark, sum of marks, and average mark.
10. Stop.

**PROGRAM CODE**

```
# Create a list of student marks
marks = [85, 92, 78, 88, 95, 67, 90]
# Display the complete list
print("Student Marks:")
print(marks)
# Total number of marks
total_students = len(marks)
print("\nTotal Number of Marks:", total_students)
# Highest mark
highest_mark = max(marks)
print("Highest Mark:", highest_mark)
# Lowest mark
lowest_mark = min(marks)
print("Lowest Mark:", lowest_mark)
# Sum of marks
total_marks = sum(marks)
```

```
print("Sum of Marks:", total_marks)
# Average mark
average_mark = total_marks / total_students
print("Average Mark:", average_mark)
```

**PROCEDURE**

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py`.
5. Run the program.

   ```
   python <filename>.py
   ```
6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

**RESULTS**

Thus, the Python program was successfully developed and executed to analyze student marks using built-in functions. The program displayed the list of marks, determined the highest and lowest marks, calculated the total and average marks, and demonstrated the effective use of Python's built-in functions for data analysis.

**CONCLUSION**

We learned how to use Python built-in functions to perform common operations efficiently without writing complex code

**PRACTICAL RELATED QUESTIONS**

1. ***Give examples of built-in functions.***

   *print(), input(), len(), max(), min(), sum(), type()*

2. ***What does the len() function return?***

   *It returns the number of elements in a sequence or collection.*

3. ***What does the type() function return?***

   *It returns the data type of a variable or object.*

4. ***Can len() be used with strings?***

   *Yes, it returns the number of characters in the string.*

5. ***Can max() and min() be used with strings?***

   *Yes, they return values based on alphabetical order.*

**SELF- LEARNING COMPONENTS**

Explore Python libraries used for Image Processing,Data Analysis, Visualization, and Machine Learning.

**ADDITIONAL PROBLEM STATEMENTS**

A cricket coach wants to analyze the scores of players using built-in functions.

1. Create a list of player scores.
2. Display all scores.
3. Find the highest score.
4. Find the lowest score.
5. Calculate the total runs scored.
6. Find the average score.
7. Display the score analysis

**REFERENCES**

| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |

| Experiment no. : 12 | Date: |
|---|---|
| **User Defined Functions in Python** | |

## PROBLEM STATEMENTS

An electricity board wants to automate the calculation of monthly electricity bills for its customers. Develop a Python program using user-defined functions with positional and default arguments to calculate the bill amount.

1. Accept the customer's name and the number of electricity units consumed.
2. Create a user-defined function calculate_bill(units) using a positional argument to calculate the electricity charge based on a fixed rate per unit.
3. Create another user-defined function final_bill(amount, service_charge=100) using a default argument to add a service charge to the bill.
4. Call the functions and calculate the electricity bill.
5. Display the electricity charge and the final bill amount.
6. Display the customer details and bill summary.

## PRACTICAL SIGNIFICANCE

*Functions are widely used in software development to divide complex programs into smaller, manageable modules. User-defined functions allow programmers to create customized solutions for specific requirements. Functions are extensively used in banking systems, inventory management, payroll processing, data analysis, and web applications to improve code reusability and reduce redundancy.*

## RELEVANT PROGRAM OUTCOMES (POs)

**PO2** – Identify and analyze well-defined engineering problems and reach substantiated conclusions using codified methods of analysis.

**PO3** – Design solutions for well-defined technical problems and assist in the design of systems, components, or processes.

**PO5** – Apply appropriate techniques, resources, and modern computing, engineering and IT tools to well-defined engineering problems.

**PO10 –** Demonstrate awareness of engineering management principles as a member or leader of a technical team and apply them to manage projects in multidisciplinary environments.

**RELEVANT COURSE OUTCOMES (COs)**

**CO4** - Design and develop modular programs using functions, modules, and packages with proper scope management and argument passing techniques.

**THEORY**

A **function** is a block of organized, reusable code that performs a specific task. Functions help reduce code repetition, improve readability, and make programs easier to maintain. Python provides two main types of functions: **Built-in Functions** and **User-Defined Functions**.

**User-defined functions** are functions created by the programmer using the def keyword. They are used when a specific operation needs to be performed repeatedly within a program. A user-defined function may accept arguments and return values.

```
def function_name(parameters):

    # Function body

    statements

    return value
```

The first component is the **function definition**, which is created using the def keyword followed by the function name and parentheses. The function definition specifies what the function will do. The second component is **parameters**, which are variables listed inside the parentheses of the function definition and are used to receive values from the function call. The third component is the **function body**, which contains the set of statements that define the operations performed by the function. Another important component is the **function call**, which executes the function by using its name and passing the required arguments. The values supplied during the function call are known as **arguments**. Finally, a function may contain a **return statement**, which sends a result back to the calling program

User-defined functions can be classified into different types based on whether they accept arguments and return values. A **function without arguments and without a**

**return value** performs a specific task without receiving any input or returning any result; it simply executes the statements inside the function body. A **function with arguments and without a return value** accepts input values from the user or program but displays the result directly instead of returning it. A **function without arguments but with a return value** does not receive any input from the caller but processes data internally and returns a result. A **function with arguments and with a return value** is the most commonly used type, where the function accepts input values, processes them, and returns the result to the calling program. In addition, Python supports **positional arguments**, where arguments are passed according to their order, and **default arguments**, where parameters are assigned predefined values that are used if no value is provided during the function call. These different types of functions provide flexibility and help programmers create efficient and reusable code for various applications.

## ALGORITHM/FLOWCHART

1. Start.
2. Input the customer's name.
3. Input the number of electricity units consumed.
4. Define a user-defined function calculate_bill(units) that calculates the electricity charge using a fixed rate per unit.
5. Define another user-defined function final_bill(amount, service_charge=100) that adds a default service charge to the bill amount.
6. Call calculate_bill(units) and store the electricity charge.
7. Call final_bill(amount) and store the final bill amount.
8. Display the customer name, units consumed, electricity charge, service charge, and final bill amount.
9. Stop.

## PROGRAM CODE

```
# User-Defined Functions with Positional and Default Arguments
# Function with positional argument
def calculate_bill(units):
    rate_per_unit = 8
    return units * rate_per_unit

# Function with default argument
def final_bill(amount, service_charge=100):
```

```
    return amount + service_charge
# Input customer details
customer_name = input("Enter Customer Name: ")
units = int(input("Enter Units Consumed: "))
# Function calls
electricity_charge = calculate_bill(units)
bill_amount = final_bill(electricity_charge)

# Display bill details
print("\n----- Electricity Bill Summary -----")
print("Customer Name      :", customer_name)
print("Units Consumed     :", units)
print("Electricity Charge :", electricity_charge)
print("Service Charge     : 100")
print("Final Bill Amount  :", bill_amount)
```

## PROCEDURE

1. Open the Python IDE.
2. Create a new Python file.
3. Enter the Python code for the problem
4. Save the file as `<filename>.py.`
5. Run the program.

```
python <filename>.py
```

6. Verify that the output is correct.
7. If errors occur, edit the code, save the changes, and run the program again.
8. Repeat the testing process with different input values.
9. Record the output and result.

## RESULTS

Thus, the Python program was successfully developed and executed using user-defined functions with positional and default arguments to calculate electricity bills. The experiment demonstrated modular programming, code reusability, and efficient problem-solving using functions.

## CONCLUSION

Learned how to create and use user-defined functions with positional and default arguments in Python. User-defined functions help in modular programming, improve code reusability, and make programs easier to understand and maintain.

## PRACTICAL RELATED QUESTIONS

1. ***What is the purpose of a function?***

   *To perform a specific task and avoid code repetition.*

2. ***Which keyword is used to define a function?***

   *The def keyword is used to define a function.*

3. ***What is a default argument?***

   *An argument that has a predefined value used when no value is passed during the function call.*

4. ***What is the purpose of the return statement?***

   *It sends a value back from a function to the calling program.*

5. ***Can a function return multiple values?***

   *Yes, a function can return multiple values separated by commas.*

## SELF- LEARNING COMPONENTS

1. Write user-defined functions in Python that use parameters and return values [Example Like]
   - ○ Factorial of a Number
   - ○ Celsius to Fahrenheit Converter
2. Explore and differentiate between positional and default arguments in Python functions, supported by examples
3.

## ADDITIONAL PROBLEM STATEMENTS

A company wants to calculate employee salaries using user-defined functions.

1. Accept employee name and basic salary.
2. Create a function calculate_salary() using a positional argument to calculate annual salary.
3. Create a function final_salary() with a default bonus amount.
4. Display employee salary details.
5. Display the final salary.

## REFERENCES


| Sl. No. | Text Book/ Website | Author | Publication |
|---|---|---|---|
| 1 | Python 3 for Absolute Beginners | Tim Hall & J P Stacey | Apress |
| 2 | Introduction to Python Programming | Gowrishankar S, Veena A | CRC Press/Taylor & Francis |
| | Web references | | |
| 3 | https://www.w3schools.com/python/ | | |
| 4 | https://nptel.ac.in/courses/106106182 | | |